

Integral sums of reciprocals of distinct squarefree semiprimes: forty-seven terms are necessary, and the extremal representations of 1

Abstract

Let $\mathcal{P} = \{pq : p < q \text{ primes}\}$ be the set of squarefree semiprimes. We prove that if $T \subset \mathcal{P}$ is a nonempty finite set such that $\sum_{n \in T} 1/n$ is an integer, then $|T| \geq 47$. Since 47-term representations of 1 are known, 47 is the exact minimum; this confirms a conjecture of Watanabe. We also classify the extremal case: there are exactly 23 sets $T \subset \mathcal{P}$ with $|T| = 47$ and $\sum_{n \in T} 1/n = 1$. Seventeen of them involve only primes at most 73, and each of the other six involves exactly one prime larger than 73, namely 269, 311, 421, 433, 503 or 3779. The proofs are computer-assisted. A p -adic argument turns integrality into a congruence condition at every vertex of a graph on the primes. A decomposition into a “core” of small primes and a “large-prime part”, together with value, parity and star-size lemmas and an explicit divisor equation for a single edge between two large primes, reduces the problem to a finite exhaustive computation, even though arbitrarily large prime factors are allowed. The enumeration of the cores uses a new pruning bound, a Lagrangian relaxation of the value condition combined with a knapsack over residues, which reduces the size of the search tree by a factor of about 3000 for 47 terms. The computation uses exact integer arithmetic and directed rounding only. With the same method we also determined all 48-term representations of 1: there are exactly 620 of them. We describe independent checks: a second implementation of the completion step, two implementations of the core enumeration with identical outputs, brute-force validation on reduced instances, and exact verification of all solutions. The analytic reduction and the 23 solutions are verified in Lean 4 with Mathlib. The core enumeration is written as a Lean function, and Lean proves that this search is exhaustive: its output contains the core of every solution. The remaining completion step enters the Lean proofs as explicitly stated hypotheses about the cores in that output; these hypotheses are established by computation and are not verified in Lean.

Contents

1	Introduction	2
2	Preliminaries and size bounds	5
3	The graph of a set of semiprimes and the local criterion	6
4	Core decomposition and the finite reduction	6
5	Large primes: stars and the large-large component	9
5.1	Stars	9
5.2	One edge between two large primes	10
6	The core search and the proof of the lower bound	11
6.1	Step 1: enumeration of the admissible cores	11
6.2	The loss bound	12
6.3	Step 2: completion of the surviving cores	15

7	Classification of the 47-term representations	16
7.1	Correspondence between the claims and the programs	18
8	Forty-eight terms	19
8.1	Two large-large elements	19
8.2	The computation	19
9	Exact and independent verification	20
9.1	Exact verification of the 23 solutions	21
9.2	Independent implementation of Step 2	21
9.3	Re-executions and the budget-46 cross-check	21
9.4	Validation of Step 1 by brute force on reduced instances	22
9.5	The search as a Lean function	23
9.6	Self-tests	23
9.7	What is not independently verified	24
10	Lean formalization	24
11	Computational reproducibility	26
12	Discussion	28
13	Disclosure of the use of generative AI	29
A	The 23 representations of 1 with 47 terms	31
B	The computational hypotheses in Lean	32

1 Introduction

An *Egyptian fraction* is a finite sum of distinct unit fractions. Many classical questions ask which rationals are sums of reciprocals of distinct integers from a prescribed set; see [4–6] and [7, Section D11]. This paper studies the set

$$\mathcal{P} = \{pq : p < q \text{ primes}\} = \{6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, \dots\}$$

of *squarefree semiprimes*, i.e. the integers with exactly two distinct prime factors, each to the first power. For a finite set $T \subset \mathcal{P}$ write

$$\Sigma(T) = \sum_{n \in T} \frac{1}{n}.$$

Background. According to the abstract of Watanabe’s preprint [14], in 1978 Johnson found a representation of 1 as a sum of reciprocals of 48 distinct squarefree semiprimes [8]. No shorter representation was known until Watanabe [14] found 17 representations with 47 terms. Watanabe found no representation with fewer than 47 terms and conjectured that 47 is the minimum. His examples were obtained by computer searches in which the prime factors were bounded.¹ A search with bounded prime factors cannot establish minimality, because arbitrarily

¹The search space described in [14] has dimension $325 = \binom{26}{2}$, the number of squarefree semiprimes whose prime factors are both at most 101. We could not consult the full text while preparing this manuscript, so this detail should be checked against [14].

large primes are allowed in principle; [Section 5](#) shows that primes as large as 3779 really do occur.

In a different direction, Butler, Erdős and Graham [1] proved that every positive integer is a sum of reciprocals of distinct integers that each have exactly three distinct prime factors, and they conjectured the corresponding statement for two prime factors. A recent preprint of Li [9] announces a proof of that conjecture. Egyptian fractions whose denominators are supported on a fixed finite set of primes are studied computationally in [2]. None of these works addresses the minimal number of terms for the set $\mathcal{P}$. To the best of our knowledge, no proof that 47 is minimal, and no complete list of the 47-term representations of 1 in which every prime is allowed, has appeared before.

Results.

Theorem A (Minimum number of terms). *Let $T \subset \mathcal{P}$ be a nonempty finite set such that $\Sigma(T) \in \mathbb{Z}$. Then $|T| \geq 47$. The bound is sharp: there are sets $T \subset \mathcal{P}$ with $|T| = 47$ and $\Sigma(T) = 1$.*

Theorem B (The extremal representations). *There are exactly 23 sets $T \subset \mathcal{P}$ with $|T| = 47$ and $\Sigma(T) = 1$. They are listed in [Appendix A](#). Equivalently, there are exactly 23 sets $T \subset \mathcal{P}$ with $|T| = 47$ and $\Sigma(T) \in \mathbb{Z}$. Of these 23 sets:*

- (i) 17 involve only primes at most 73 (in fact at most 71);
- (ii) each of the other 6 involves exactly one prime $P > 73$, where P is one of 269, 311, 421, 433, 503 and 3779. This prime occurs in exactly three elements aP, bP, cP , where $a, b, c \leq 73$ are primes with $P \mid ab + ac + bc$ (see [Table 2](#)).

In particular, the 17 solutions of (i) are exactly the solutions all of whose prime factors are at most 101. Our 17 solutions of type (i) have the same count as Watanabe’s list of 17 examples. We have not compared the two lists term by term.

The method is not limited to 47 terms. Run with budget 48, the same search and completion determine all representations of 1 by 48 distinct squarefree semiprimes.

Theorem C (Forty-eight terms). *There are exactly 620 sets $T \subset \mathcal{P}$ with $|T| = 48$ and $\Sigma(T) = 1$ (equivalently, $\Sigma(T) \in \mathbb{Z}$). They are listed in the file `solutions48.txt` of the repository. Of these 620 sets:*

- (i) 397 involve only primes at most 73;
- (ii) 187 involve exactly one prime $P > 73$, all of whose neighbours are at most 73;
- (iii) 18 involve exactly two primes $P, P' > 73$, neither of which divides an element of T together with the other;
- (iv) 18 involve exactly two primes $P_1, P_2 > 73$ and contain the element $P_1 P_2$.

[Theorem C](#) rests on one further computational ingredient, the treatment of shapes with two large-large elements ([Section 8](#)), which is not needed for [Theorems A](#) and [B](#). The largest prime that occurs in a 48-term representation is 1210883 ([Section 8](#)).

Nature of the proof. All three theorems are *computer-assisted*. [Sections 2](#) to [5](#) give a complete mathematical reduction of the statements to finite computations. We state the computations for [Theorems A](#) and [B](#) precisely as [Computational Claims 6.4](#) and [7.1](#). The computation has two steps: the enumeration of all possible “cores” (Step 1) and their completion

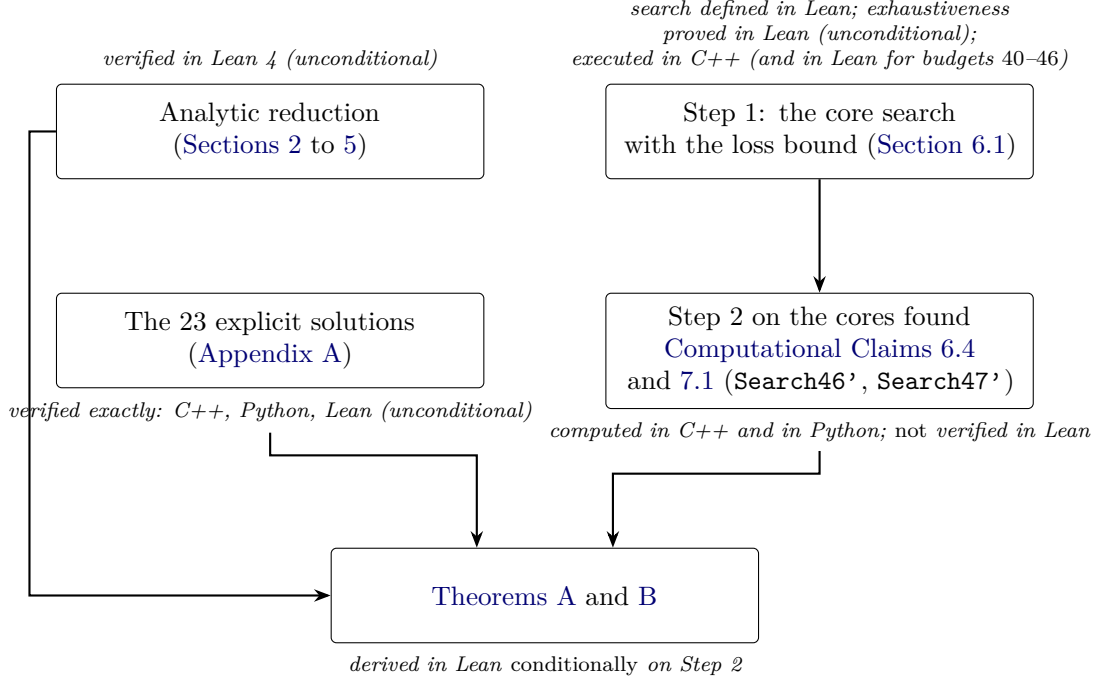


Figure 1: Logical structure of the proofs of [Theorems A](#) and [B](#) and the verification status of each component. [Theorem C](#) has the same structure, with an additional completion routine for two large-large elements ([Section 8](#)); its Step 2 is not stated in Lean.

(Step 2). Step 1 is carried out by a depth-first search with a new pruning bound, the *loss bound* ([Section 6.2](#)); it is written as a Lean function, and Lean proves that the search is exhaustive ([Section 10](#)). Both steps were executed by the C++ program `semiprime48.cpp`, which uses exact integer arithmetic and directed rounding only ([Sections 6 to 8 and 11](#)). An earlier implementation without the loss bound (`semiprime_reciprocals.cpp` and `semiprime47.cpp`) produced identical results for [Theorems A](#) and [B](#). [Section 9](#) describes the independent checks, and [Section 10](#) describes the Lean 4 formalization. [Figure 1](#) summarizes the logical status of each ingredient.

Why large primes are the main difficulty. Only finitely many elements of $\mathcal{P}$ have both prime factors in a given finite set. Since $\mathcal{P}$ contains $2P$ for every prime P , however, no a priori bound on the prime factors of a solution is evident. The main theoretical step is to show that, once the “small-prime” part of a solution is fixed, the large primes are confined to an explicit finite set. A large prime all of whose neighbours are small must divide an explicit integer $n(A)$ ([Lemma 5.1](#)). A single edge joining two large primes leads to a factorization identity with finitely many solutions ([Proposition 5.5](#)). The solution with the prime $3779 = 29 \cdot 37 + 29 \cdot 41 + 37 \cdot 41$ shows that such large primes really occur.

Organization. [Section 2](#) gives the size bounds, and [Section 3](#) the graph-theoretic form of integrality. [Sections 4 and 5](#) develop the finite reduction. [Section 6](#) describes the core search, including the loss bound, and proves [Theorem A](#). [Sections 7 and 8](#) prove [Theorems B](#) and [C](#). [Sections 9 to 11](#) cover verification, formalization and reproducibility. [Section 12](#) discusses limitations, and [Section 13](#) discloses the use of generative AI.

2 Preliminaries and size bounds

Let $a_1 < a_2 < a_3 < \dots$ be the elements of $\mathcal{P}$ in increasing order, and let

$$H_k = \sum_{i=1}^k \frac{1}{a_i} \quad (k \geq 0).$$

The first 47 elements of $\mathcal{P}$ are 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 86, 87, 91, 93, 94, 95, 106, 111, 115, 118, 119, 122, 123, 129, 133, 134, 141, 142, 143, 145, 146, 155, 158.

Lemma 2.1 (Exchange lemma). *Let $\mathcal{Q} = \{c_1 < c_2 < \dots\} \subset \mathbb{Z}_{>0}$ be infinite. If $T \subset \mathcal{Q}$ is finite with $|T| \leq k$, then $\Sigma(T) \leq \sum_{i=1}^k 1/c_i$. In particular $\Sigma(T) \leq H_k$ for every $T \subset \mathcal{P}$ with $|T| \leq k$.*

Proof. Write $T = \{t_1 < \dots < t_m\}$ with $m \leq k$. The elements $t_1, \dots, t_i$ are i distinct elements of $\mathcal{Q}$, so $t_i \geq c_i$. Hence $\Sigma(T) = \sum_{i \leq m} 1/t_i \leq \sum_{i \leq m} 1/c_i \leq \sum_{i \leq k} 1/c_i$. $\square$

Proposition 2.2 (Exact values). *The values of H_k in Table 1 are exact reduced fractions. In particular*

$$H_{35} < 1 - \frac{1757}{79000}, \quad H_{37} < 1 < H_{38}, \quad H_{46} < H_{47} < H_{48} < 2.$$

Table 1: Exact values of H_k . Each inequality in the last column can be read off by comparing numerator and denominator. The decimal expansions (truncated) are for orientation only.

	a_k	numerator	denominator	decimal
H_{35}	119	14455695312142253588	14790464231955481839	$0.977365\dots < 1 - \frac{1757}{79000}$
H_{37}	123	1793055508868583336521	1804436636298568784358	$0.993692\dots < 1$
H_{38}	129	1807043389770122629423	1804436636298568784358	$1.001444\dots > 1$
H_{46}	155	94992310862397653755543	89805923752009843120899	$1.057751\dots < 2$
H_{47}	158	15098591040010839136496693	14189335952817555213102042	$1.064080\dots < 2$

Proof. This is a finite exact computation with rational numbers. It was carried out independently in three ways: with the big-integer routines of the C++ programs (Section 11), in Python with `Fractions.Fraction` (script `paper/verification/verify_paper_facts.py`), and in Lean 4, where the inequalities are proved by `norm_num` (`H35_val`, `H37_lt_one`, `H38_gt_one`, `H46_lt_two`, `H47_lt_two`; Section 10). Lean also proves that $a_1, \dots, a_{47}$ are the 47 smallest elements of $\mathcal{P}$. Finally $a_{48} = 159$ and $H_{48} = H_{47} + 1/159 < 1.0704$ (checked with exact rationals by `paper/verification/verify48_facts.py`; `semiprime48.cpp` checks $H_{48} < 2$ exactly). $\square$

Corollary 2.3. *Let $T \subset \mathcal{P}$ be nonempty with $|T| \leq 48$ and $\Sigma(T) \in \mathbb{Z}$. Then $\Sigma(T) = 1$ and $|T| \geq 38$.*

Proof. By Lemma 2.1 and Proposition 2.2, $0 < \Sigma(T) \leq H_{48} < 2$, so $\Sigma(T) = 1$. If $|T| \leq 37$ then $\Sigma(T) \leq H_{37} < 1$, a contradiction. $\square$

In Lean, the case $|T| \leq 47$ is `recipSum_eq_one_of_integral`. For $|T| \leq 48$ (`recipSum_eq_one_of_integral_48`) Lean uses the cruder bound $\Sigma(T) \leq H_{47} + 1/6 < 3/2 + 1/6$, obtained by removing one element. Consequently, Theorem A is equivalent to the statement that no set of at most 46 elements of $\mathcal{P}$ has reciprocal sum 1. Likewise, the 47-element sets with integral sum in Theorem B, and the 48-element sets with integral sum in Theorem C, are exactly those with sum 1.

3 The graph of a set of semiprimes and the local criterion

Definition 3.1. For a finite $T \subset \mathcal{P}$ let $\Gamma(T)$ be the simple graph whose vertices are the primes dividing some element of T and whose edges are the pairs $\{p, q\}$ with $pq \in T$. For a prime p let $N(p) = N_T(p) = \{q : pq \in T\}$ be its set of neighbours ($N(p) = \emptyset$ if p is not a vertex).

The map $T \mapsto \Gamma(T)$ is a bijection between finite subsets of $\mathcal{P}$ and finite simple graphs on the set of primes without isolated vertices. The number of edges is $|T|$. For a prime p and an integer m with $p \nmid m$ we write m^{-1} for the inverse of m modulo p .

Lemma 3.2 (Local criterion). *For every finite $T \subset \mathcal{P}$,*

$$\Sigma(T) \in \mathbb{Z} \iff \sum_{q \in N(p)} q^{-1} \equiv 0 \pmod{p} \text{ for every prime } p. \quad (1)$$

Proof. Let ν_p denote the p -adic valuation on $\mathbb{Q}$. A rational number is an integer if and only if its p -adic valuation is nonnegative for every prime p . Fix p and split

$$\Sigma(T) = \frac{1}{p} A_p + B_p, \quad A_p = \sum_{q \in N(p)} \frac{1}{q}, \quad B_p = \sum_{n \in T, p \nmid n} \frac{1}{n}.$$

Every element of T is squarefree, so every denominator in B_p and in A_p is prime to p , and $\nu_p(B_p) \geq 0$. Write $A_p = u/v$ with $v = \prod_{q \in N(p)} q$ and $u = \sum_{q \in N(p)} v/q$. Then $p \nmid v$ and $u \equiv v \sum_{q \in N(p)} q^{-1} \pmod{p}$. If the congruence in (1) holds at p , then $p \mid u$, so $\nu_p(A_p/p) \geq 0$ and hence $\nu_p(\Sigma(T)) \geq 0$. Conversely, if $\nu_p(\Sigma(T)) \geq 0$, then $\nu_p(A_p/p) = \nu_p(\Sigma(T) - B_p) \geq 0$, so $p \mid u$ and the congruence holds. For a prime p that divides no element of T both sides are trivially true. $\square$

Corollary 3.3. *If $\Sigma(T) \in \mathbb{Z}$, then no vertex of $\Gamma(T)$ has degree 1.*

Proof. If $N(p) = \{q\}$ then $q^{-1} \not\equiv 0 \pmod{p}$. $\square$

For every element of the 23 solutions of [Theorem B](#), the congruences (1) were checked directly at every vertex, as an independent test of the implementation. In Lean the lemma is `integral_iff_localCond`. Its proof uses the common denominator $\prod_{p \in \Gamma(T)} p$ instead of valuations.

4 Core decomposition and the finite reduction

Throughout this section $K \in \{46, 47, 48\}$ is a *budget*, and $T \subset \mathcal{P}$ is a finite set with

$$\Sigma(T) = 1, \quad |T| \leq K. \quad (2)$$

By [Lemma 3.2](#), the congruence (1) holds at every prime.

Definition 4.1 (Small and large primes). Let $\mathcal{S} = \{2, 3, 5, \dots, 73\}$ be the set of the 21 primes at most 73. Call the primes $P > 73$ *large*; the smallest large primes are $b_0 = 79 < b_1 = 83 < b_2 = 89 < \dots$. Let $\mathcal{P}_{\mathcal{S}} = \{pq \in \mathcal{P} : p, q \in \mathcal{S}\}$, a set of $\binom{21}{2} = 210$ elements.

The primes dividing $a_1, \dots, a_{46}$ are exactly the elements of $\mathcal{S}$ (since $146 = 2 \cdot 73 \leq a_{46} = 155 < a_{47} = 158 = 2 \cdot 79$). The reduction below is valid for any finite cutoff; the choice of $\mathcal{S}$ only affects the efficiency of the computation.

Definition 4.2 (Core, large part, residues). For T as in (2) define:

- the *core* $C = T \cap \mathcal{P}_S$ and the *large part* $G = T \setminus C$;
- for $q \in \mathcal{S}$ the *core residue* $\rho(q) = \sum_{r: qr \in C} r^{-1} \in \mathbb{Z}/q\mathbb{Z}$;
- the *unsatisfied set* $U = U(C) = \{q \in \mathcal{S} : \rho(q) \neq 0\}$;
- for $q \in \mathcal{S}$ the number d_q of large neighbours of q ;
- the number b of *large-large* elements of G , i.e. elements with both prime factors large.

Every element of G is either qP with $q \in \mathcal{S}$ and P large, or PP' with P, P' both large. In particular every element of G has at most one prime factor in $\mathcal{S}$. The quantities ρ and U depend only on C .

Lemma 4.3 (Unsatisfied primes). *Every $q \in U$ has a large neighbour. Hence $|G| \geq |U|$ and*

$$|C| + |U| \leq |T| \leq K.$$

Proof. At $q \in \mathcal{S}$ the congruence (1) reads $\rho(q) + \sum_P P^{-1} \equiv 0 \pmod{q}$, where P runs over the large neighbours of q . If $\rho(q) \neq 0$, this sum is nonempty. Choosing one element $e(q) = qP \in G$ for each $q \in U$ gives an injection $U \rightarrow G$, because an element of G has at most one prime factor in $\mathcal{S}$. $\square$

For $x \geq 0$ let $V(x)$ be the sum of the x largest values of $1/n$ over the elements $n \in \mathcal{P}$ that have a large prime factor. Thus $V(x) = \frac{1}{158} + \frac{1}{166} + \frac{1}{178} + \frac{1}{194} + \frac{1}{202} + \frac{1}{206} + \dots$ (x terms). Since every such n is at least $2 \cdot 79 = 158$, we have $V(x) \leq x/158$.

Lemma 4.4 (Value bound, general form). *For T as in (2) we have $\Sigma(C) \leq 1$. Moreover, for every subset $U' \subseteq U$,*

$$1 \leq \Sigma(C) + \sum_{q \in U'} \frac{1}{79q} + V(K - |C| - |U'|). \quad (3)$$

Proof. First, $\Sigma(C) = 1 - \Sigma(G) \leq 1$. Take the elements $e(q)$, $q \in U'$, from the proof of Lemma 4.3. They are distinct, and $e(q) = qP \geq 79q$. The remaining $|G| - |U'| \leq K - |C| - |U'|$ elements of G are distinct elements of $\mathcal{P}$ with a large prime factor. By Lemma 2.1 their reciprocal sum is at most $V(K - |C| - |U'|)$. Now use $1 = \Sigma(C) + \Sigma(G)$. $\square$

Definition 4.5 (Admissible cores). A set $C \subseteq \mathcal{P}_S$ is K -admissible if $|C| + |U(C)| \leq K$ and $\Sigma(C) \leq 1$, and if (3) holds for every upper segment $U' = U(C) \cap (p, \infty)$, $p \geq 0$. (The upper segments include $U(C)$ itself, for $p = 0$, and $\emptyset$, for $p \geq 73$.)

By Lemmas 4.3 and 4.4, the core of every T satisfying (2) is K -admissible.

Remark 4.6 (Why upper segments). The search of Section 6 processes the primes of $\mathcal{S}$ in decreasing order, and at an intermediate stage it uses (3) with U' equal to the set of primes *already known* to lie in U . That set is an upper segment of U . A prime that joins U later, for instance $q = 2$ at the last level, adds $1/(79q)$ to the right-hand side of (3) but removes one term of V , possibly a much smaller one. So the right-hand side for $U' = U$ can be larger than for a proper upper segment. Thus the set of cores produced by the search is *not* the set of cores satisfying (3) for $U' = U$ alone, as stated in an earlier description of the method. It contains the set of admissible cores in the sense of Definition 4.5, and possibly some further cores, because the bounds used at intermediate nodes are not tight. Pruning with upper segments is sound because (3) holds for every $U' \subseteq U$. On reduced instances the difference is substantial (Section 9.4).

Definition 4.7 (Shapes). Let C be a core with residues ρ and unsatisfied set U . For $q \in U$ let $P_{\min}(q)$ be the least large prime P with $P^{-1} \equiv -\rho(q) \pmod{q}$. Put

$$w_q(0) = 0, \quad w_q(1) = \frac{1}{q P_{\min}(q)}, \quad w_q(d) = \sum_{k=0}^{d-1} \frac{1}{q b_k} \quad (d \geq 2).$$

A pair (d, b) , with $d = (d_q)_{q \in \mathcal{S}}$ and b nonnegative integers, is a K -feasible shape for C if

(S1) $d_q \geq 1$ for $q \in U$ and $d_q \neq 1$ for $q \in \mathcal{S} \setminus U$;

(S2) $E := b + \sum_{q \in U} (d_q - 1) + \sum_{q \in \mathcal{S} \setminus U} d_q \leq K - |C| - |U|$;

(S3) $\rho(2) + d_2 \equiv 0 \pmod{2}$, where $\rho(2) \in \{0, 1\}$;

(S4) $1 - \Sigma(C) \leq \sum_{q \in \mathcal{S}} w_q(d_q) + \frac{b}{79 \cdot 83}$.

Dirichlet's theorem guarantees that $P_{\min}(q)$ exists, but we do not need it. If no large prime lies in the required class, then $d_q = 1$ is impossible by [Lemma 4.8](#) below. The programs compute $P_{\min}(q)$ by direct search, and this search terminated for every class that occurred. The Lean formalization defines $P_{\min}(q) = 0$ if no such prime exists.

Lemma 4.8 (Shape lemma). *For T as in (2), the pair (d, b) of [Definition 4.2](#) is a K -feasible shape for the core C of T .*

Proof. (S1): For $q \in U$ use [Lemma 4.3](#). If $q \notin U$ has exactly one large neighbour P , then (1) at q gives $P^{-1} \equiv 0 \pmod{q}$, which is impossible.

(S2): Each element of G has exactly one small prime factor or none, so $|G| = \sum_{q \in \mathcal{S}} d_q + b$. Hence $E = |G| - |U| \leq K - |C| - |U|$.

(S3): Every large prime P is odd, so $P^{-1} \equiv 1 \pmod{2}$. The congruence (1) at 2 becomes $\rho(2) + d_2 \equiv 0 \pmod{2}$.

(S4): Split

$$1 - \Sigma(C) = \Sigma(G) = \sum_{q \in \mathcal{S}} \sum_{P \text{ large}, qP \in T} \frac{1}{qP} + \sum_{PP' \in G \text{ large-large}} \frac{1}{PP'}.$$

If $d_q \geq 2$, the d_q large neighbours of q are distinct primes, so their contribution is at most $w_q(d_q)$ by [Lemma 2.1](#). If $d_q = 1$, then $q \in U$ by (S1). By (1) the unique large neighbour P satisfies $P^{-1} \equiv -\rho(q)$, so $P \geq P_{\min}(q)$ and $1/(qP) \leq w_q(1)$. A large-large element is PP' with $79 \leq P < P'$, so $P' \geq 83$ and $1/(PP') \leq 1/(79 \cdot 83)$. $\square$

The finite reduction. [Lemmas 4.3, 4.4](#) and [4.8](#) reduce the problem as follows.

- (1) The core is one of finitely many admissible subsets of the 210 elements of $\mathcal{P}_{\mathcal{S}}$.
- (2) The shape is one of finitely many, because all d_q and b are bounded by (S2).
- (3) The large part is then subject to the constraints of [Section 5](#): a large prime whose neighbours are all small divides an explicit positive integer, and a single large-large edge is governed by a factorization identity. Both confine the large primes to explicit finite sets.

The only case not reduced by theory is $b \geq 2$. The computation shows that no feasible shape with $b \geq 2$ exists for $K = 47$, and none with $b \geq 1$ for $K = 46$ ([Sections 6](#) and [7](#)). For $K = 48$, feasible shapes with $b = 2$ do occur, and they are handled by the additional argument of [Section 8.1](#); no feasible shape with $b \geq 3$ exists. The programs were written to report every shape they cannot handle as “unresolved” rather than discard it.

5 Large primes: stars and the large-large component

5.1 Stars

For a finite set A of primes put

$$n(A) = \sum_{q \in A} \prod_{r \in A \setminus \{q\}} r,$$

so that $\sum_{q \in A} 1/q = n(A) / \prod_{r \in A} r$. For instance $n(\{a, b\}) = a + b$ and $n(\{a, b, c\}) = ab + ac + bc$.

Lemma 5.1 (Star lemma). *Let P be a prime with $P \notin A$. Then $\sum_{q \in A} q^{-1} \equiv 0 \pmod{P}$ if and only if $P \mid n(A)$. Consequently, if $\Sigma(T) \in \mathbb{Z}$ and a large prime P has all its neighbours in $\mathcal{S}$, then $P \mid n(N(P))$ and in particular $P \leq n(N(P))$. Moreover $|N(P)| \geq 2$.*

Proof. Multiply the congruence by $\prod_{r \in A} r$, which is invertible modulo P . The second statement follows from (1) at P and $n(A) > 0$; the third follows from Corollary 3.3. $\square$

We call such a pair (P, A) with $A = N(P) \subseteq \mathcal{S}$ a *star* with *centre* P and *leaves* A . For a fixed set A of leaves only the finitely many prime divisors of $n(A)$ can be centres. This is the mechanism that bounds the large primes.

Lemma 5.2 (Star size). *Suppose T satisfies (2) and P is a large prime all of whose neighbours lie in $\mathcal{S}$. Then $|N(P)| \leq |T| - 36 \leq K - 36$. Thus a star has at most 10 leaves if $K = 46$, at most 11 if $K = 47$ and at most 12 if $K = 48$.*

Proof. Let $d = |N(P)|$. The d elements qP contribute at most $\frac{1}{79} \sum_{q \in \mathcal{S}} \frac{1}{q} \leq \frac{1757}{79000}$, since $\sum_{q \in \mathcal{S}} 1/q = 1.7565 \dots \leq 1.757$ (exact computation). If $d \geq |T| - 35$, the other at most 35 elements contribute at most $H_{35} < 1 - \frac{1757}{79000}$ by Lemma 2.1 and Proposition 2.2. Then $\Sigma(T) < 1$, a contradiction. $\square$

The programs use the slightly sharper bound $\frac{1}{79} \sum_{i < d} 1/p_i + H_{K-d} \geq 1$, with $p_0 < p_1 < \dots$ the primes of $\mathcal{S}$. Recomputed with exact rationals, it gives the same maxima, 10, 11 and 12.

Definition 5.3 (Candidate stars). Let C be a core and (d, b) a feasible shape, and put $W = \{q \in \mathcal{S} : d_q \geq 1\}$. A *candidate star* for (C, d) with at most m leaves is a pair (P, A) with $A \subseteq W$, $2 \leq |A| \leq m$, $P > 73$ prime, $P \mid n(A)$, and $P^{-1} \equiv -\rho(q) \pmod{q}$ for every $q \in A$ with $d_q = 1$.

Since $P \leq n(A) \leq |A| \prod_{r \in A} r \leq 21 \prod_{r \in \mathcal{S}} r$, there are only finitely many candidate stars (`starCand_finite`). If $b = 0$, every large prime of T is the centre of a star, and the stars of T have the following properties:

- their centres are distinct;
- $q \in \mathcal{S}$ is a leaf of exactly d_q of them;
- each of them is a candidate star with at most $K - 36$ leaves (Lemmas 5.1 and 5.2). The residue condition holds because a leaf q with $d_q = 1$ has the centre as its only large neighbour;
- their total value $\sum_{(P,A)} \sum_{q \in A} 1/(qP)$ equals $1 - \Sigma(C)$.

Conversely, these conditions determine G . So for $b = 0$ the completions of a core are exactly the *exact covers* of the multiplicities (d_q) by candidate stars with distinct centres and total value $1 - \Sigma(C)$.

The six solutions with a large prime. Table 2 lists the six solutions of Theorem B that contain a large prime. In each of them G consists of a single star with three leaves a, b, c . The core has 44 elements, $U = \{a, b, c\}$, $d_a = d_b = d_c = 1$ and all other $d_q = 0$, so the excess is $E = 0$ and $|C| + |U| = 47$. The large prime divides $n(\{a, b, c\}) = ab + ac + bc$. For $P = 311$, 503 and 3779 the quantity $ab + ac + bc$ is itself prime and equal to P ; for $P = 269$, 421 and 433 it is $3P$, $11P$ and $7P$, respectively.

Table 2: The six 47-term representations of 1 that contain a prime $P > 73$. The last column refers to the numbering of Appendix A.

P	leaves $A = \{a, b, c\}$	$n(A) = ab + ac + bc$	large elements aP, bP, cP	no.
269	$\{5, 11, 47\}$	$807 = 3 \cdot 269$	1345, 2959, 12643	11
311	$\{3, 5, 37\}$	311	933, 1555, 11507	2
421	$\{29, 43, 47\}$	$4631 = 11 \cdot 421$	12209, 18103, 19787	12
433	$\{3, 37, 73\}$	$3031 = 7 \cdot 433$	1299, 16021, 31609	1
503	$\{3, 13, 29\}$	503	1509, 6539, 14587	3
3779	$\{29, 37, 41\}$	3779	109591, 139823, 154939	5

Example 5.4 (The prime 3779). Solution 5 of Appendix A contains $109591 = 29 \cdot 3779$, $139823 = 37 \cdot 3779$ and $154939 = 41 \cdot 3779$, and $3779 = 29 \cdot 37 + 29 \cdot 41 + 37 \cdot 41$ is prime. Any search that bounds the prime factors by a fixed cutoff below 3779 misses this representation of 1. Such a cutoff includes the natural choices 101 and 1000. The present argument bounds large primes only through divisibility: a star centre divides $n(A)$ for its set of leaves A , which for three leaves can be as large as $67 \cdot 71 + 67 \cdot 73 + 71 \cdot 73 = 14831$.

5.2 One edge between two large primes

Suppose $b = 1$. Then G contains exactly one element $P_1 P_2$ with $P_1 \neq P_2$ both large. Every other large prime has only small neighbours, since a second large neighbour would create a second large-large element, and is therefore the centre of a star. For $i = 1, 2$ we have $N(P_i) = A_i \cup \{P_{3-i}\}$ with $A_i \subseteq \mathcal{S}$, and $A_i \neq \emptyset$ by Corollary 3.3. Put

$$b_i = \prod_{q \in A_i} q, \quad a_i = n(A_i), \quad Z = \sum_{q \in A_1} \frac{1}{qP_1} + \sum_{q \in A_2} \frac{1}{qP_2} + \frac{1}{P_1 P_2},$$

the value of the component. Finally put

$$W = a_1 b_2 P_2 + a_2 b_1 P_1 + b_1 b_2, \quad t = Z b_1 b_2, \quad N = b_1 b_2 (t + a_1 a_2).$$

Proposition 5.5 (The component equation). *Let $P_1 \neq P_2$ be primes not dividing $b_1 b_2$, and let A_1, A_2 be nonempty.*

- (a) $Z b_1 b_2 P_1 P_2 = W$; that is, $t P_1 P_2 = W$ in $\mathbb{Q}$.
- (b) The local conditions (1) at P_1 and at P_2 hold if and only if $P_1 P_2 \mid W$, i.e. if and only if $t \in \mathbb{Z}$.
- (c) For every rational $t \neq 0$,

$$(tP_1 - a_1 b_2)(tP_2 - a_2 b_1) = b_1 b_2 (t + a_1 a_2) \iff tP_1 P_2 = W.$$

In particular the identity on the left holds for $t = Z b_1 b_2$, whether or not t is an integer.

- (d) Suppose t is a positive integer with $tP_1P_2 = W$, and put $X = tP_1 - a_1b_2$, $Y = tP_2 - a_2b_1$. Then X and Y are positive integers with $XY = N$, and

$$P_1 = \frac{X + a_1b_2}{t}, \quad P_2 = \frac{N/X + a_2b_1}{t}.$$

Conversely, if $t > 0$ and X is a positive divisor of N such that both quotients are integers, then they satisfy $tP_1P_2 = W$.

Proof. (a) Use $\sum_{q \in A_i} 1/q = a_i/b_i$ and clear denominators.

(b) Since $N(P_1) = A_1 \cup \{P_2\}$, [Lemma 5.1](#) shows that (1) at P_1 is equivalent to $P_1 \mid n(A_1 \cup \{P_2\}) = a_1P_2 + b_1$. Now $W = b_2(a_1P_2 + b_1) + a_2b_1P_1 \equiv b_2(a_1P_2 + b_1) \pmod{P_1}$ and $P_1 \nmid b_2$, so this is equivalent to $P_1 \mid W$. The same argument applies at P_2 . Since $P_1 \neq P_2$, both conditions together are equivalent to $P_1P_2 \mid W$, which by (a) says $t \in \mathbb{Z}$.

(c) Expanding gives $(tP_1 - a_1b_2)(tP_2 - a_2b_1) - b_1b_2(t + a_1a_2) = t(tP_1P_2 - W)$.

(d) From $tP_1P_2 = W$ we get $P_2X = W - a_1b_2P_2 = a_2b_1P_1 + b_1b_2 > 0$, so $X > 0$, and similarly $Y > 0$. Then (c) gives $XY = N$. The converse is (c) read backwards. $\square$

Remark 5.6 (Correction). An earlier description of the method stated that the factorization identity in (c) is itself equivalent to the local conditions at P_1 and P_2 . By (a) and (c), however, the identity holds automatically for $t = Zb_1b_2$. What is equivalent to the local conditions is the *integrality* of t , by (b). The positivity of X and Y and the nonemptiness of A_1 and A_2 , both used by the program, are proved above.

Finite enumeration for $b = 1$. Fix a core C , a feasible shape with $b = 1$, nonempty attachment sets $A_1, A_2 \subseteq W$, and a family F of stars covering the remaining multiplicities $d_q - [q \in A_1] - [q \in A_2]$. Then the value of the component is $Z = 1 - \Sigma(C) - \Sigma(F)$, and $t = Zb_1b_2$ is determined; it must be a positive integer. By [Proposition 5.5\(d\)](#) the pair (P_1, P_2) is then determined by a positive divisor X of the explicit integer N . All choices of A_1, A_2 and F range over finite sets, so the $b = 1$ completions of a core form a finite, explicitly enumerable set (`comp_pairs_finite`). Interchanging (A_1, P_1) and (A_2, P_2) replaces X by N/X , so unordered pairs $\{A_1, A_2\}$ suffice.

6 The core search and the proof of the lower bound

This section describes the computation and proves [Theorem A](#) from its outcome for $K = 46$. The computation consists of two steps: enumeration of all admissible cores (Step 1) and, for each of them, enumeration of all feasible shapes and completions (Step 2). Both steps are carried out by the program `semiprime48.cpp` for any budget $K \leq 48$. Its Step 1 is the depth-first search of the earlier programs `semiprime_reciprocals.cpp` (budget 46) and `semiprime47.cpp` (budget 47), with one additional pruning test, the loss bound (P4) of [Section 6.2](#). That test reduces the number of search nodes by a factor of about 5000 for budget 46 and 3000 for budget 47, and it does not change the output ([Lemma 6.3](#)).

6.1 Step 1: enumeration of the admissible cores

Let $p_0 = 2 < p_1 < \dots < p_{20} = 73$ be the primes of $\mathcal{S}$. Every core $C \subseteq \mathcal{P}_{\mathcal{S}}$ is determined by the sets

$$L_j = \{p_i : i < j, p_i p_j \in C\} \quad (j = 20, 19, \dots, 1)$$

of *lower neighbours*. The search is a depth-first search that chooses $L_{20}, L_{19}, \dots, L_1$ in this order (Algorithm 1). When L_j is chosen, every core edge at p_j is known, because the edges to larger primes were fixed at earlier levels. Hence $\rho(p_j)$ is known, and it is decided whether $p_j \in U$. At every stage the set of primes already known to lie in U is therefore an upper segment $U \cap (p, \infty)$ of the final set U . Each core corresponds to exactly one leaf of the search tree, so each core is visited at most once.

For each j , the choice of L_j is split into two parts. The primes p_i with $8 \leq i < j$ are handled by explicit inclusion/exclusion recursion. The 8 smallest primes are handled by a precomputed table of all 2^8 subsets, indexed by their contribution to the residue at p_j . This lets the search jump directly to the subsets that make p_j satisfied, and to the others (which put p_j into U) only when that is still affordable.

Fixed-point bounds. Write $\text{up}(n) = \lceil 2^{96}/n \rceil$ and $\text{lo}(n) = \lfloor 2^{96}/n \rfloor$. For a finite set X of positive integers put $\text{up}(X) = \sum_{n \in X} \text{up}(n)$ and $\text{lo}(X) = \sum_{n \in X} \text{lo}(n)$. Then

$$\text{lo}(X) \leq 2^{96} \Sigma(X) \leq \text{up}(X).$$

Let $v_1 < v_2 < \dots$ be the elements of $\mathcal{P}$ with a large prime factor, and put $\text{VB}(x) = \sum_{i \leq x} \text{up}(v_i) \geq 2^{96} V(x)$. All these quantities are integers below 2^{128} and are computed exactly in unsigned 128-bit arithmetic.

Pruning tests. Consider a node of the search at which the decided core edges form a set C_0 and the decided unsatisfied primes form U_0 . Let j be the current level and i the number of lower primes whose membership in L_j is still open. The search discards the node if one of the following holds:

$$(P1) \quad |C_0| + |U_0| > K;$$

$$(P2) \quad \text{lo}(C_0) > 2^{96};$$

$$(P3) \quad \text{up}(C_0) + \sum_{q \in U_0} \text{up}(79q) + B_{j,i}(R) < 2^{96}, \text{ where } R = K - |C_0| - |U_0| \text{ and}$$

$$B_{j,i}(R) = \max_{\substack{x_1 + x_2 \leq R \\ x_1 \leq i}} (M_{j,i}(x_1) + M'_j(x_2) + \text{VB}(R - x_1 - x_2)).$$

Here $M_{j,i}(x)$ is the sum of the x largest values $\text{up}(p_j p_k)$ with $k < i$ (possible further edges at p_j), and $M'_j(x)$ is the sum of the x largest values $\text{up}(p_k p_l)$ with $k < l < j$ (possible edges among smaller primes).

Test (P3) is also applied to the tentative set $U_0 \cup \{p_j\}$, with R replaced by $R - 1$, before p_j is put into U . The fourth test, (P4), is described in Section 6.2. At a leaf, where the complete core C and U are known, the core is kept only if (P1) and (P2) hold and

$$\text{Val}(C) := \text{up}(C) + \sum_{q \in U} \text{up}(79q) + \text{VB}(K - |C| - |U|) \geq 2^{96}. \quad (4)$$

6.2 The loss bound

Why the search without (P4) is slow. Profiling the search with (P1)–(P3) only shows that almost all nodes lie in branches in which some prime has been put into U . For instance, for budget 45 the search visits $1.07 \cdot 10^{10}$ nodes, but only $2.9 \cdot 10^7$ when unsatisfied primes are switched off (the latter is not a valid search; it only shows where the nodes are). Once a

Algorithm 1 Step 1 (core enumeration), schematically

```

1: procedure LEVEL( $j, C_0, U_0$ )                                ▷ primes  $> p_j$  fully processed
2:   if  $j = 0$  then                                           ▷ prime 2: no lower neighbours
3:     decide  $2 \in U$  from  $\rho(2)$ ; apply (P1); LEAF( $C_0, U_0$ ); return
4:   for all  $L \subseteq \{p_0, \dots, p_{j-1}\}$  not excluded by (P1)–(P4) do    ▷ explicit + table
5:      $C_1 \leftarrow C_0 \cup \{p_j q : q \in L\}$ ; compute  $\rho(p_j)$  and the residue updates at  $q \in L$ 
6:      $U_1 \leftarrow U_0 \cup \{p_j\}$  if  $\rho(p_j) \neq 0$  else  $U_0$           ▷ (P3) tested for  $U_1$ 
7:     LEVEL( $j - 1, C_1, U_1$ )
8: procedure LEAF( $C, U$ )
9:   if (4) holds then output  $C$  to Step 2
  
```

prime is in U , every set of neighbours is allowed at it, and the value bound (P3) ignores the congruences (1) that the primes not yet processed will have to satisfy. Satisfying them forces a loss of value that (P3) cannot see, so such branches die only many levels further down. Test (P4) makes this loss visible, prime by prime.

Weights. All quantities are integers in units of 2^{-96} . Fix $\theta = \lfloor 2^{96}/145 \rfloor$ (any $\theta \geq 0$ would be valid; this value was tuned by experiment). For an edge $e = p_a p_b$ with $a < b$ and for $q \in \mathcal{S}$ put

$$w(e) = \text{up}(p_a p_b) - \theta, \quad g(q) = \text{up}(79q) - \theta, \quad V_\theta = \max_{0 \leq x \leq K} (\text{VB}(x) - x\theta).$$

If C is a core with $|C| + |U| \leq K$ and $x = K - |C| - |U|$, then

$$\text{Val}(C) = K\theta + \sum_{e \in C} w(e) + \sum_{q \in U} g(q) + (\text{VB}(x) - x\theta) \leq K\theta + V_\theta + \sum_{e \in C} w(e) + \sum_{q \in U} g(q). \quad (5)$$

The weights may be negative; θ acts as a Lagrange multiplier for the budget constraint.

Shares. The weight of an edge $e = p_a p_b$, $a < b$, is split between its endpoints. With $\sigma_a = s_a/12$, where $(s_0, \dots, s_5) = (12, 11, 11, 10, 10, 9)$ and $s_a = 8$ for $a \geq 6$, the larger endpoint receives $\text{sh}_b(e) = \lceil \sigma_a w(e) \rceil$ and the smaller one $\text{sh}_a(e) = \lceil (1 - \sigma_a)w(e) \rceil$. Then $\text{sh}_a(e) + \text{sh}_b(e) \geq w(e)$. (Any $\sigma_a \in [0, 1]$ is valid; these values were tuned by experiment.)

Local bounds. Consider a prime p_k whose membership in U is not yet decided, let X be the set of its possible further neighbours, and let r be the residue at p_k of its decided edges. In a final core, either $p_k \in U$, or the set $Y \subseteq X$ of its further neighbours satisfies $r + \sum_{y \in Y} y^{-1} \equiv 0 \pmod{p_k}$. Hence the shares received by p_k from its further edges, plus $g(p_k)$ if $p_k \in U$, total at most

$$\text{loc}(k; X, r) = \max \left(g(p_k) + \sum_{x \in X} \max(\text{sh}_k(p_k x), 0), \max_{\substack{Y \subseteq X \\ r + \sum_{y \in Y} y^{-1} \equiv 0 \pmod{p_k}}} \sum_{y \in Y} \text{sh}_k(p_k y) \right).$$

The inner maximum is a knapsack over the residues modulo p_k . At a node of the search, X depends only on the level (j, i) and on k , so every value of loc that can occur is a precomputed table entry indexed by the level, k and r (about 20 000 entries).

(P4) At a node with decided core edges C_0 and decided unsatisfied primes U_0 , the search

discards the node if

$$\text{up}(C_0) + \sum_{q \in U_0} \text{up}(79q) + (K - |C_0| - |U_0|)\theta + V_\theta + \sum_k \text{loc}(k; X_k, r_k) < 2^{96},$$

where k runs over the primes not yet decided, X_k is the set of their possible further neighbours and r_k the residue of their decided edges.

The sum over k changes by a bounded amount when an edge is added or rejected, and for the table-driven step a 256-entry array gives it for all subsets of the 8 smallest primes at once. All quantities are integers, and the shares are rounded up.

Proposition 6.1 (Soundness of the loss bound). *For every core C with $|C| + |U(C)| \leq K$ in the subtree of a node, the left-hand side of (P4) is at least $\text{Val}(C)$. In particular (P4) never discards a node above a core that passes (4).*

Proof. Split $C = C_0 \cup C_1$ and $U = U_0 \cup U_1$ into the decided and undecided parts. Then $\sum_{C_0} w + \sum_{U_0} g = \text{up}(C_0) + \sum_{U_0} \text{up}(79q) - (|C_0| + |U_0|)\theta$, so by (5) it suffices to show $F := \sum_{e \in C_1} w(e) + \sum_{q \in U_1} g(q) \leq \sum_k \text{loc}(k; X_k, r_k)$. Every edge of C_1 joins two undecided primes, and $w(e) \leq \text{sh}_a(e) + \text{sh}_b(e)$. Hence F is at most the sum over the undecided primes p_k of the shares that p_k receives from its edges in C_1 , plus $g(p_k)$ if $p_k \in U_1$. By the choice of loc , the contribution of p_k is at most $\text{loc}(k; X_k, r_k)$. $\square$

Lemma 6.2 (Soundness of Step 1). *Every K -admissible core (Definition 4.5) reaches a leaf of the search and is passed to Step 2.*

Proof. Let C be admissible, with unsatisfied set U , and consider a node on the path to C , with $C_0 \subseteq C$ and $U_0 = U \cap (p, \infty)$ for some p . (P1) fails because $|C_0| + |U_0| \leq |C| + |U| \leq K$. (P2) fails because $\text{lo}(C_0) \leq 2^{96}\Sigma(C_0) \leq 2^{96}\Sigma(C) \leq 2^{96}$. For (P3), let x_1 and x_2 be the numbers of edges of $C \setminus C_0$ at p_j and among the smaller primes, respectively. Then $\text{up}(C \setminus C_0) \leq M_{j,i}(x_1) + M'_j(x_2)$. Also $R - x_1 - x_2 = K - |C| - |U_0|$, and $\text{VB}(x) \geq 2^{96}V(x)$. Hence

$$\text{up}(C_0) + \sum_{q \in U_0} \text{up}(79q) + B_{j,i}(R) \geq 2^{96} \left(\Sigma(C) + \sum_{q \in U_0} \frac{1}{79q} + V(K - |C| - |U_0|) \right) \geq 2^{96},$$

where the last step is (3) for the upper segment $U' = U_0$. The tentative test with $U_0 \cup \{p_j\}$ is again a test of this form for the upper segment $U \cap [p_j, \infty)$, and (4) is the case $U' = U$; in particular $\text{Val}(C) \geq 2^{96}$. Finally (P4) fails by Proposition 6.1. $\square$

Because of the rounding, the search may keep a few cores that are not admissible. This is harmless: they are examined further in Step 2. Lemma 6.2 is proved in Lean for the search written as a Lean function (`core_mem_search`, Section 10).

Lemma 6.3 (Same output). *The search with (P1)–(P4) outputs exactly the same cores as the search with (P1)–(P3) only.*

Proof. Adding a test can only remove cores. Conversely, a core output by the search without (P4) passes (4), so by Proposition 6.1 test (P4) does not discard any node above it. $\square$

In agreement with Lemma 6.3, the archived core lists of the earlier programs for budgets 46 and 47 are reproduced line for line by `semiprime48.cpp` (Section 9.3).

Parallelization. The search tree above a fixed level (the prime 43 in `semiprime48.cpp`, 61 in the earlier programs) is traversed by every thread in the same deterministic order, and the nodes at that level are numbered consecutively. Each thread obtains node numbers from a shared atomic counter and processes only the nodes whose numbers it obtained. Tickets are handed out in increasing order, and a thread requests a new ticket only after passing the node of its previous ticket. Hence the ticket a thread obtains is never smaller than its current position, and every node number below the total count is obtained by exactly one thread. So every subtree is processed exactly once, regardless of scheduling. Only the reported node counts depend on the number of threads, because the upper part of the tree is traversed by every thread.

6.3 Step 2: completion of the surviving cores

For each core C output by Step 1:

- (a) If $U = \emptyset$, then C satisfies (1) at every prime: at the primes of $\mathcal{S}$ by the definition of U , and trivially at all other primes. So $\Sigma(C) \in \mathbb{Z}$. If $C \neq \emptyset$, then $\Sigma(C) = 1$ by Corollary 2.3, and the program reports C as a solution. The empty core is not admissible, because $V(K) \leq K/158 < 1$.
- (b) If $U \neq \emptyset$, the program enumerates all (d, b) satisfying (S1) and (S2), discards those that violate (S3) or (S4), and processes the survivors. In (S4) the right-hand side is rounded up and $1 - \Sigma(C)$ is rounded down, so no feasible shape is discarded. For a surviving shape with $b = 0$, the program lists all candidate stars with at most $K - 36$ leaves (Definition 5.3); it finds the centres by factoring $n(A)$ completely. It then enumerates the exact covers of the multiplicities by candidate stars with distinct centres whose total value is $1 - \Sigma(C)$. Surviving shapes with $b = 1$ and $b = 2$ are treated in Sections 7 and 8.1; a surviving shape with $b \geq 3$ is reported as *unresolved*.

For budget 46 the earlier program `semiprime_reciprocals.cpp` only searched for one cover satisfying (1) at every $q \in W$ and reported every shape with $b > 0$ as unresolved; for this budget the difference does not matter, because no candidate star and no shape with $b > 0$ occurs.

Computational Claim 6.4 (Budget 46; Lean: `Search46`, `Search46'`). *For every 46-admissible core C :*

- (i) $U(C) \neq \emptyset$;
- (ii) every 46-feasible shape (d, b) for C has $b = 0$;
- (iii) for every 46-feasible shape $(d, 0)$ there is no candidate star with at most 10 leaves.

The run of `semiprime48.cpp` with budget 46 produced the output summarized in Table 3. The search visited $2.6 \cdot 10^7$ nodes and kept 178 cores; none of them has $U = \emptyset$. Step 2 examined 3777 shapes satisfying (S1)–(S3). Only 61 of them also satisfy (S4), all with $b = 0$, and for none of these 61 shapes does a candidate star exist. Nothing was reported as unresolved and no solution was found. This is precisely Computational Claim 6.4; see Section 7.1 for the correspondence in detail. The recorded run of the earlier program `semiprime_reciprocals.cpp`, which visited $1.4 \cdot 10^{11}$ nodes, kept the same 178 cores and printed the same Step 2 counts; it was repeated for the first version of this paper with identical results (Section 9.3).

Proof of Theorem A, assuming Computational Claim 6.4. Let $T \subset \mathcal{P}$ be nonempty with $|T| \leq 46$ and $\Sigma(T) \in \mathbb{Z}$. By Corollary 2.3, $\Sigma(T) = 1$, so T satisfies (2) with $K = 46$. Its core C is 46-admissible (Lemmas 4.3 and 4.4), so $U \neq \emptyset$ by Computational Claim 6.4(i). The shape (d, b) of T is 46-feasible (Lemma 4.8), so $b = 0$ by Computational Claim 6.4(ii). Choose $q \in U$. By

Table 3: Recorded results of `semiprime48.cpp` with 4 threads (`validation48/run46_output.txt`, `validation48/run47_output.txt`, `run48_output.txt`). For budgets 46 and 47 the earlier programs (`proof_run_output.txt`, `run47_output.txt`) printed the same values for all quantities they report; they did not report the split by b (for budget 47 it was obtained at the time with the Python program of [Section 9.2](#), with the same result). Their node counts and times are given in the last rows.

	budget 46	budget 47	budget 48
maximal number of leaves of a star (Lemma 5.2)	10	11	12
search nodes	26 112 391	598 305 041	13 344 629 391
cores kept by Step 1	178	22 382	1 491 334
of which $U = \emptyset$	0	17	414
of which $U \neq \emptyset$ and $ C + U = K$	111	11 287	702 715
shapes satisfying (S1)–(S3)	3 777	1 107 702	116 874 859
shapes satisfying (S1)–(S4)	61	8 049	601 685
cores with at least one such shape	61	7 432	509 055
with $b = 0$	61	7 981	592 627
with $b = 1$	0	68	8 985
with $b = 2$	0	0	73
with $b \geq 3$	0	0	0
candidate stars (summed over surviving shapes)	0	125	24 065
unresolved cores	0	0	0
probable (not proven) primes used	0	0	0
solutions with $ T = K$	0	23	620
wall-clock time (search and completion)	2.1 s	65 s	29 min 30 s
search nodes without (P4) (earlier programs)	139 830 520 211	1 795 181 713 099	–
wall-clock time of the earlier programs	19 min 40 s	5 h 48 min	–

[Lemma 4.3](#), q has a large neighbour P . Since $b = 0$, all neighbours of P lie in $\mathcal{S}$. By [Lemmas 5.1](#) and [5.2](#), $(P, N(P))$ satisfies $2 \leq |N(P)| \leq 10$ and $P \mid n(N(P))$, and $N(P) \subseteq W$. Moreover, if $d_{q'} = 1$ for some $q' \in N(P)$, then P is the only large neighbour of q' , so $P^{-1} \equiv -\rho(q') \pmod{q'}$ by (1). Thus $(P, N(P))$ is a candidate star for (C, d) , contradicting [Computational Claim 6.4\(iii\)](#). Hence $|T| \geq 47$. Sharpness follows from [Theorem B](#), or directly from any of the sets in [Appendix A](#), whose validity is verified exactly and unconditionally ([Section 9.1](#)). $\square$

7 Classification of the 47-term representations

For $K = 47$ the program `semiprime48.cpp` runs Step 1 as in [Section 6](#). Its Step 2 enumerates *all* completions, as did Step 2 of the earlier program `semiprime47.cpp`:

- cores with $U = \emptyset$ are recorded as solutions;
- for a surviving shape with $b = 0$, all exact covers by candidate stars with at most 11 leaves and distinct centres are enumerated, and a cover is kept if its value equals $1 - \Sigma(C)$ *exactly*. The value is computed as an integer multiple of $1/D$, where $D = \prod_{p \in \mathcal{S}} p < 2^{128}$ is a common denominator of all core and star values (the value of a star (P, A) is $(n(A)/P)/\prod_{q \in A} q$);
- for a surviving shape with $b = 1$, the program enumerates the following, as in [Section 5.2](#):
 - all unordered pairs $\{A_1, A_2\}$ of nonempty subsets of W with $[q \in A_1] + [q \in A_2] \leq d_q$;
 - all covers of the remaining multiplicities by candidate stars;
 - the integer t , if it exists;

- all positive divisors X of N , obtained from the complete factorization of $t + a_1a_2$ and of b_1b_2 .

It keeps the pairs for which $P_1 \neq P_2$ are primes larger than 73 and distinct from the star centres;

- a surviving shape with $b = 2$ is treated as in [Section 8.1](#) (in `semiprime47.cpp`: reported as unresolved), and a surviving shape with $b \geq 3$ is reported as unresolved.

Every recorded set is then verified exactly: it has 47 elements, they are distinct squarefree semiprimes, and their reciprocal sum is 1 ([Section 9.1](#)).

Computational Claim 7.1 (Budget 47; Lean: `Search47`, `Search47'`). *Let $\mathcal{T}_{23}$ be the family of the 23 sets listed in [Appendix A](#).*

- (i) *Every 47-admissible core $C \neq \emptyset$ with $U(C) = \emptyset$ belongs to $\mathcal{T}_{23}$.*
- (ii) *For every 47-admissible core with $U \neq \emptyset$, every 47-feasible shape has $b \leq 1$.*
- (iii) *($b = 0$) Let C be 47-admissible with $U \neq \emptyset$, let $(d, 0)$ be a 47-feasible shape, and let $\{(P, A_P)\}_{P \in C}$ be a family of candidate stars with at most 11 leaves and distinct centres such that each $q \in \mathcal{S}$ is a leaf of exactly d_q of them and $\Sigma(C) + \sum_P \sum_{q \in A_P} 1/(qP) = 1$. Then $C \cup \{qP : P \in C, q \in A_P\} \in \mathcal{T}_{23}$.*
- (iv) *($b = 1$) Let C and $(d, 1)$ be as in (iii), except that $b = 1$. Suppose we are given:*
 - *nonempty $A_1, A_2 \subseteq \mathcal{S}$;*
 - *a family of candidate stars as in (iii) covering the multiplicities $d_q - [q \in A_1] - [q \in A_2]$, with total value $\Sigma_F < 1 - \Sigma(C)$;*
 - *the positive integer $t = (1 - \Sigma(C) - \Sigma_F) b_1 b_2$;*
 - *a positive divisor X of N such that $P_1 = (X + a_1 b_2)/t$ and $P_2 = (N/X + a_2 b_1)/t$ are distinct primes larger than 73 and are not centres of the stars.*

Then the resulting set, consisting of C , the star elements, $\{qP_1\}_{q \in A_1}$, $\{qP_2\}_{q \in A_2}$ and $P_1 P_2$, belongs to $\mathcal{T}_{23}$.

The run of `semiprime48.cpp` and the recorded run of `semiprime47.cpp` ([Table 3](#)) kept the same 22 382 cores. Seventeen of them have $U = \emptyset$, all with $|C| = 47$, and these are the 17 solutions with all primes at most 73. Of the remaining 22 365 cores, 11 287 have $|C| + |U| = 47$. The printed count “ $|C| + |U| = 47$: 11 287” in `run47_output.txt` excludes the 17 cores with $U = \emptyset$. Including them, the dump of all kept cores contains 11 304 cores with $|C| + |U| = 47$. Step 2 examined 1 107 702 shapes. Of these, 8 049 shapes in 7 432 cores satisfy (S1)–(S4): 7 981 with $b = 0$, 68 with $b = 1$, and none with $b \geq 2$. For the 68 shapes with $b = 1$ there are 7 896 admissible attachment pairs; t is an integer in 957 cases, and none of them yields a solution. The surviving shapes with $b = 0$ have 125 candidate stars in total. They produce exactly the 6 solutions of [Table 2](#), each consisting of a core with 44 elements and a single star with three leaves. No shape was unresolved. In the run of `semiprime47.cpp` every number declared prime is below 2^{64} , where the primality test is deterministic, and “composite” verdicts of the Miller–Rabin test are always correct; `semiprime48.cpp` reports 0 probable primes. So all primality decisions are rigorous ([Section 11](#)). Both programs found exactly the same 23 solutions.

Proof of Theorem B, assuming Computational Claim 7.1. By [Section 9.1](#), each of the 23 listed sets has 47 elements and reciprocal sum 1, and the sets are pairwise distinct. Conversely, let $|T| = 47$ and $\Sigma(T) = 1$. Then (2) holds with $K = 47$, and the core C of T is 47-admissible.

- If $U = \emptyset$, then $G = \emptyset$. Indeed, C satisfies (1) everywhere, so $\Sigma(C) \in \mathbb{Z}$. Since $V(47) < 1$, the core C is nonempty, so $\Sigma(C) = 1$ and $\Sigma(G) = 0$. Hence $T = C \in \mathcal{T}_{23}$ by [Computational Claim 7.1\(i\)](#).
- If $U \neq \emptyset$, the shape of T is feasible, so $b \leq 1$ by [Computational Claim 7.1\(ii\)](#).
 - If $b = 0$, the stars of T form a family as in [Computational Claim 7.1\(iii\)](#) ([Section 5](#)), so $T \in \mathcal{T}_{23}$.
 - If $b = 1$, [Proposition 5.5](#) shows that the data $(A_1, A_2, \text{stars}, t, X)$ of T are as in [Computational Claim 7.1\(iv\)](#), with $X = tP_1 - a_1b_2 > 0$. Hence $T \in \mathcal{T}_{23}$.

The statement about integral sums follows from [Corollary 2.3](#). Parts (i) and (ii) of [Theorem B](#) are read off from the list; the numbers in [Table 2](#) were recomputed exactly. $\square$

Structure of the 23 solutions. All 23 solutions contain the 23 numbers 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 93, 95, 133, 155. Each solution involves between 14 and 16 distinct primes, and the union of the 23 solutions has 122 elements. The largest prime in the 17 solutions of type (i) is 71. [Table 4](#) lists, for each solution, its largest prime, its core size, and its elements outside the common part.

7.1 Correspondence between the claims and the programs

[Computational Claims 6.4](#) and [7.1](#) are stated with exact rational quantities, and the programs implement them as follows.

- **Step 1.** By [Lemma 6.2](#), every admissible core is output by the search. For the search written as a Lean function this is a theorem of the formalization ([Section 10](#)). That the C++ program computes the same function is not proved; it is supported by the comparisons of [Section 9.5](#), where the compiled Lean function reproduces the C++ output.
- **Shapes.** The shape enumeration generates every (d, b) satisfying (S1) and (S2). Since (S4) is tested with a rounded-up right-hand side, every feasible shape passes.
- **Candidate stars.** For every $A \subseteq W$ with $2 \leq |A| \leq m$, the program removes all prime factors at most 73 from $n(A)$ and factors the remainder completely. Each factor is certified prime by a deterministic test, and the product of the factors equals the remainder by construction. So every candidate star of [Definition 5.3](#) is found. The residue condition for $d_q = 1$ is tested exactly.
- **Covers.** Covers are enumerated over index-increasing sequences of candidates with distinct centres, so every family of [Computational Claim 7.1\(iii\)](#) is produced exactly once.
- **Case $b = 1$.** The program uses unordered pairs $\{A_1, A_2\}$, whereas [Computational Claim 7.1\(iv\)](#) quantifies over ordered pairs. Swapping $(A_1, P_1) \leftrightarrow (A_2, P_2)$ replaces X by N/X and yields the same set.
- **Magnitudes.** In `semiprime47.cpp` all intermediate integers in the $b = 1$ branch are stored in 128-bit words without explicit overflow checks. The Python recount ([Section 9.2](#)) shows that in the actual budget-47 run the largest of them, N , has at most 59 bits. Moreover, b_1b_2 , a_1b_2 and a_2b_1 have at most 31 bits and $t + a_1a_2$ at most 28 bits. So no overflow occurred. In `semiprime48.cpp` all star and component arithmetic is 128-bit with explicit overflow checks, and an overflow would be reported as unresolved.

This correspondence is an informal argument about the source code. It is *not* machine-checked; see [Section 10](#).

8 Forty-eight terms

With budget $K = 48$ the same two steps determine all sets $T \subset \mathcal{P}$ with $|T| \leq 48$ and $\Sigma(T) = 1$. Step 1 is the search of [Section 6](#), which is exhaustive by [Lemma 6.2](#); in Lean this is `core_mem_searchCores` with $K = 48$. Step 2 is that of [Section 7](#), with stars of at most 12 leaves ([Lemma 5.2](#)), and with one additional routine for shapes with $b = 2$.

8.1 Two large-large elements

Suppose that T satisfies (2) and that its shape has $b = 2$. Every large prime that divides no large-large element is the centre of a star, as in [Section 5.2](#). The two large-large elements either share a prime, forming a path $P_0 - P_1 - P_2$, or they are disjoint, forming two edges P_0P_1 and P_2P_3 . For each of these large primes P_i let $A_i = N(P_i) \cap \mathcal{S}$ be its set of small neighbours, and let $a_i = n(A_i)$ and $b_i = \prod_{q \in A_i} q$ (with $a_i = 0$, $b_i = 1$ if $A_i = \emptyset$). By [Corollary 3.3](#), $A_i \neq \emptyset$ for every P_i except the middle prime P_1 of a path. The value of these elements is

$$Z = 1 - \Sigma(C) - \Sigma(F) = \sum_i \frac{a_i}{b_i P_i} + \sum_{PP'} \frac{1}{PP'},$$

where F is the set of star elements and the last sum runs over the two large-large elements. This is a sum of at most 5 positive terms for a path and at most 6 for two edges. Hence one term is at least $Z/5$ or $Z/6$, respectively, and therefore some P_i satisfies $P_i \leq 6a_i/(b_i Z)$, or some large-large element PP' satisfies $\min(P, P') \leq \sqrt{6/Z}$. For fixed C , attachment sets and stars, Z is known, so at least one of the large primes is bounded by an explicit number B (the program uses the sharper constant 5 for paths).

The program tries every prime p with $73 < p \leq B$ in every position:

- if p is the middle of a path, then the two ends have the neighbours $A_0 \cup \{p\}$ and $A_2 \cup \{p\}$, so by [Lemma 5.1](#) they are prime factors of $n(A_0 \cup \{p\}) = pa_0 + b_0$ and of $pa_2 + b_2$;
- if p is an end of a path, the other two primes form a single large-large edge in which p is one more neighbour of the middle prime, and they are found through the divisor equation of [Proposition 5.5](#);
- if p lies on one of two disjoint edges, its partner is a prime factor of $pa + b$ for the attachment data (a, b) of the partner, and the other edge is found through the divisor equation.

A candidate is accepted only if its exact value is Z , and every accepted set is then verified exactly. Since in every completion some large prime is at most B , every completion with $b = 2$ is found. This argument is not formalized in Lean.

8.2 The computation

The run of `semiprime48.cpp` with budget 48 is summarized in the last column of [Table 3](#). Step 1 kept 1 491 334 cores, 414 of them with $U = \emptyset$. Step 2 examined 116 874 859 shapes; 601 685 of them, in 509 055 cores, satisfy (S1)–(S4): 592 627 with $b = 0$, 8 985 with $b = 1$, 73 with $b = 2$ (in 73 cores), and none with $b \geq 3$. For the 73 shapes with $b = 2$ the program solved 62 169 problems of the above form (choices of attachment sets and star covers); the largest bound B was 1581, and none of them yields a completion. No core was unresolved and no probable prime was used; primality is decided by the Miller–Rabin test with the first 13 prime bases, which is deterministic below $3.3 \cdot 10^{24}$ [12], and every number tested lies below that bound. The run took 29 min 30 s with 4 threads.

Step 2 finds every completion G with $|G| \leq 48 - |C|$, so the run finds every T with $|T| \leq 48$ and $\Sigma(T) = 1$. It found 620 sets with 48 elements and, as a built-in consistency check, exactly the 23 sets of [Theorem B](#).

Proof of [Theorem C](#), assuming the computation. Let $|T| = 48$ and $\Sigma(T) \in \mathbb{Z}$; then $\Sigma(T) = 1$ by [Corollary 2.3](#). The core of T is 48-admissible, so it is output by Step 1 ([Lemma 6.2](#)). Its shape is feasible ([Lemma 4.8](#)), and its large part is a completion of one of the kinds enumerated by Step 2: $b \leq 2$, since no feasible shape with $b \geq 3$ occurs, and the completions with $b = 0, 1$ and 2 are enumerated completely by [Sections 5, 5.2 and 8.1](#). Hence T is one of the 620 listed sets. Conversely, each listed set has 48 elements and reciprocal sum 1 (verified exactly, see below). The classification (i)–(iv) is read off from the list. $\square$

Structure of the 620 solutions. [Table 5](#) gives the classification of [Theorem C](#); since no completion with $b = 2$ exists, every solution has at most one large-large element. The largest prime that occurs is 1 210 883, in the set that contains

$$11 \cdot 1\,210\,883, \quad 29 \cdot 1\,210\,883, \quad 37 \cdot 1\,210\,883, \quad 12\,109 \cdot 1\,210\,883, \quad 17 \cdot 12\,109;$$

its largest element $12\,109 \cdot 1\,210\,883 = 14\,662\,582\,247$ is the largest element of any of the 620 sets. Exactly 12 numbers lie in every solution: 6, 10, 14, 15, 21, 22, 26, 33, 34, 38, 39, 46. In particular, the 48-term representation of 1 attributed to Johnson [8] in [14] must be one of the 620 sets. We have not been able to consult [8] and have not identified it.

Verification. Every listed set was verified in three ways: by `semiprime48.cpp` with big-integer fractions; by `verify48.py`, with its own factorization, a deterministic primality test and Python fractions; and by `paper/verification/verify48_facts.py`, which uses trial division, checks $\Sigma(T) = 1$ with Python fractions, checks the congruences (1) at every vertex, and recomputes all numbers of this subsection. The Python program `complete48.py` is an independent implementation of Step 2 for budget 48, including its own code for $b = 2$. Run on all 1 491 334 cores of the dump `cores48.txt.gz`, it finds the same 620 + 23 sets and no unresolved core. Both implementations of the routine for $b = 2$ recover planted examples (a path and a pair of disjoint edges) with identical sets of completions. An earlier run, made before the $b = 2$ routine existed, reported exactly the 73 cores with $b = 2$ as unresolved and produced a core dump identical to that of the final run.

Status. [Theorem C](#) is computer-assisted in the same sense as [Theorems A and B](#), with two differences: its Step 2, including the routine for $b = 2$, is not stated in Lean, and the 620 sets are verified exactly by the C++ and Python programs but not in Lean. The Lean formalization covers the reduction to Step 2 for budget 48: the integrality of $\Sigma(T)$ implies $\Sigma(T) = 1$ (`recipSum_eq_one_of_integral_48`), and the core of every solution is in the output of the verified search (`core_mem_searchCores`).

9 Exact and independent verification

We distinguish five layers of evidence: (1) the optimized C++ searches, in two generations (without and with the loss bound); (2) independent Python re-implementations of Step 2; (3) exact verification of the solutions; (4) the search written as a Lean function, proved exhaustive in Lean and executed for comparison; (5) the Lean formalization of the reduction

(Section 10). We also report checks performed specifically for this paper. The verification of the 620 sets of Theorem C is described in Section 8.

9.1 Exact verification of the 23 solutions

Each of the 23 sets was verified in five independent ways.

- `semiprime47.cpp` and `semiprime48.cpp` factor every element by trial division, certify both factors with a deterministic primality test, and compute the reciprocal sum as a big-integer fraction over the product of all primes involved.
- `complete47.py` checks the sum with Python’s `fractions.Fraction`; so does `verify48.py`, which also checks that the 47-element sets found by the budget-48 run are exactly these 23.
- In Lean, `Sol23_valid` proves that each set consists of 47 squarefree semiprimes, with explicit factorizations and primality proved by `norm_num`, and has reciprocal sum exactly 1. `Sol23_card` proves that the 23 sets are pairwise distinct.
- For this paper, the script `paper/verification/verify_paper_facts.py` rechecks everything with exact integer arithmetic and trial division. It checks distinctness of the elements and of the sets, the squarefree-semiprime property, $\Sigma(T) = 1$, and the congruences (1) at every vertex. It also recomputes all numbers quoted in Sections 2 and 5 and generates all tables of this paper from the data.

9.2 Independent implementation of Step 2

The Python program `complete47.py` is a separate implementation of Step 2. It uses exact rationals (`fractions.Fraction`) and unbounded integers throughout, its own primality test and factorization (trial division followed by Pollard’s rho method [10]), and its own cover and divisor enumeration. It imposes no bound on the number of leaves of a candidate star, so it examines a superset of the candidates. It also recomputes U from each core and compares it with the value in the dump. Run on the file `cores47.txt.gz` of all 22 382 cores kept by Step 1 with $K = 47$, it finds the same 23 solutions and no unresolved core (`run47_python_crosscheck.txt`). For this paper we reran it with additional counters (`paper/verification/analyze47.py`, identical logic). It reproduces every count in the budget-47 column of Table 3: 1 107 702 shapes, of which 8 049 survive in 7 432 cores, and 125 candidate stars (with or without the bound of 11 leaves). It also gives the split by b and the magnitudes quoted in Section 7.1. The run takes about half a minute.

The Python program `complete48.py` does the same for budget 48, with its own implementation of the routine of Section 8.1 (Section 8).

Limitations. The Python programs check Step 2 only. They consume the output of the C++ Step 1 and cannot detect a core that Step 1 failed to output; that gap is addressed by Lemma 6.2, its formal version, and the comparisons of Section 9.5. They were written within the same project and implement the same mathematical reduction (Sections 4 and 5), so they are independent *implementations*, not independent *arguments*.

9.3 Re-executions and the budget-46 cross-check

Re-execution. For this paper the budget-46 program `semiprime_reciprocals.cpp` was compiled and executed again with 4 threads (GCC 13.3.0; a cloud container with 4 virtual CPUs, Intel Xeon 2.10 GHz). The output is identical to the recorded `proof_run_output.txt`,

including the node count 139 830 520 211, except for the elapsed time (1531 s, with other jobs running concurrently); log: `paper/verification/rerun/`.

Budget-46 cores and a second Step 2. The engine of `semiprime47.cpp`, included unmodified in a small driver, was run with budget 46; it writes every core kept by Step 1 to a file. It visited 139 830 520 211 nodes, kept 178 cores, examined 3 777 shapes of which 61 survive, found 0 candidate stars, 0 unresolved cores and 0 solutions, in agreement with Table 3. The Python Step 2 (`analyze47.py`, the logic of `complete47.py`) applied to these 178 cores with budget 46 likewise finds 3 777 shapes satisfying (S1)–(S3), 61 satisfying (S1)–(S4) (all with $b = 0$), 0 candidate stars (also without the bound of 10 leaves), 0 unresolved shapes and 0 solutions. This archives the Python check of the budget-46 Step 2 mentioned in the repository documentation. Since the Step 1 code of both programs is the same, this is a second *execution* of Step 1, not an independent implementation.

Re-execution of the budget-47 search. The program `semiprime47.cpp` was also executed again, with 4 threads and the option `--dump`, on the same machine. All printed quantities coincide with the recorded `run47_output.txt`: the node count 1 795 181 713 099, the 22 382 cores, the shape and candidate counts, 0 unresolved cores and 0 probable primes. The list of 23 solutions is identical, and the new core dump coincides line by line (as a set of 22 382 lines) with `cores47.txt.gz`. Only the elapsed time differs (19 388 s for the search). Files: `paper/verification/rerun47/`.

Old and new search. By Lemma 6.3 the searches with and without (P4) have the same output. This was checked directly (`validation48/`): `semiprime48.cpp` reproduces the archived core dumps of the earlier programs line for line, 178 cores for budget 46 (with $2.6 \cdot 10^7$ instead of $1.4 \cdot 10^{11}$ nodes) and 22 382 cores for budget 47 (with $6.0 \cdot 10^8$ instead of $1.8 \cdot 10^{12}$ nodes). For budgets 40–45 on the real instance both programs keep no core, and on 15 reduced instances ($\mathcal{S}$ = the primes up to 13, 17, 19 and 23, with up to 21 million cores) the two programs produce identical dumps. On these small instances (P4) prunes nothing, so they test the bookkeeping of the new code rather than the pruning itself.

9.4 Validation of Step 1 by brute force on reduced instances

Step 1 has no second full-scale implementation in C++ or Python (the two C++ generations share most of their code, and the Lean function of Section 9.5 is too slow for budgets above 46). To test the C++ code, we ran the engine of `semiprime47.cpp` on reduced instances, with $\mathcal{S}$ replaced by the primes up to 13, 17 and 19 and with various budgets. We compared its output with a brute-force enumeration of *all* 2^{15} , 2^{21} and 2^{28} cores, respectively. For this test only, a copy of the program with Step 2 disabled was used; the Step 1 code is unchanged. For every core the brute force computes two sets, using the same fixed-point rounding as the program:

- SEG: the cores satisfying (P1), $\Sigma(C) \leq 1$ exactly, and the rounded test (4) for *every* upper segment of U (Definition 4.5);
- FIN: the cores satisfying (P1), (P2) and the final test (4), i.e. only the case $U' = U$.

Lemma 6.2 and the form of the leaf test predict $\text{SEG} \subseteq D \subseteq \text{FIN}$ for the set D of cores output by the search. This was confirmed in all 48 configurations tested: 12 pairs of cutoff and budget, the default table size of 8 low primes and a reduced size of 3 (which exercises the explicit recursion), and 1 and 3 threads. The search output never contained a duplicate. A negative control, which compared a budget-45 search with the reference sets for budget 50, was correctly

reported as a failure. Representative sizes are in [Table 6](#). They also show that FIN can be much larger than SEG, which quantifies [Remark 4.6](#).

These tests exercise the same code path as the real search, including the table-driven enumeration, the residue bookkeeping, the U -accounting, all pruning tests and the parallel work distribution, on instances small enough to enumerate completely. They do not replace an independent full-scale implementation of Step 1. The same brute-force check was passed by `semiprime48.cpp` on the reduced instances with $\mathcal{S}$ = the primes up to 13, 17 and 19 (`validation48/step1_equivalence_output.txt`).

9.5 The search as a Lean function

The Lean function `Inst.search` of [Section 10](#) follows Step 1 of `semiprime48.cpp`: the same order of the primes, the same splitting into explicit recursion and a final step for the 8 smallest primes, the same tests (P1)–(P4) with the same integer tables, and the same final test. It is written for clarity, not speed: cores are finite sets of index pairs, residues are recomputed at every node, and the final step runs through all 256 subsets of the 8 smallest primes instead of using a table indexed by residues. Lean proves that its output contains every admissible core ([Section 10](#)). Its output is not certified to be *equal* to that of the C++ program. To test the correspondence we compiled the Lean function to native code and compared its output with that of `semiprime48.cpp` (option `--dump`; harness `validation48/dfs48_harness.cpp`):

- on 12 reduced instances ($\mathcal{S}$ = the primes up to 13 with budgets 40, 45, 50, 60; up to 17 and up to 19 with budgets 40, 45, 50; up to 23 with budgets 40, 45; instance generator `mkInst`), the two outputs coincide as sets of cores; the largest outputs have 274 486 cores ($\mathcal{S}$ = primes up to 23, budget 45) and 478 084 cores ($\mathcal{S}$ = primes up to 19, budget 50);
- on the real instance ($\mathcal{S}$ = the primes up to 73), for budgets 40 to 45 both keep no core, and for budget 46 both keep the same 178 cores.

The compiled Lean function is several hundred to several thousand times slower than the C++ program (for budget 46: about 8.5 CPU hours, against 4.6s for the C++ search with one thread), mainly because it recomputes all residues at every node and evaluates all 256 choices at every final step. For budget 46 it was therefore run in several processes: `search_eq_rounds` (`SearchSplit.lean`) proves that the search is the concatenation, in order, of the searches below the 890 624 work items obtained by 26 rounds of refinement of the root; the heaviest item was refined further in the same way (`flatMap_expandItem`), and every piece was evaluated by exactly one process (`validation48/lean_runner/README.md`). Budgets 47 and 48 are out of reach for the Lean function. The comparison script and its output are `validation48/lean_vs_cpp.sh` and `validation48/lean_vs_cpp_output.txt`.

9.6 Self-tests

The recorded self-tests (`selftest_output.txt`) provide further consistency checks.

- The engine, run without unsatisfied primes and with budget 47, finds exactly 17 solutions. These are the 17 solutions of [Theorem B\(i\)](#).
- The completion routine reconstructs a planted solution with the cutoff lowered to 53 and to 43. In the second case the shape has positive excess.
- The full pipeline with cutoff 53 and budget 47 finds 21 solutions, all among the 23. With cutoff 53 it also leaves 2902 unresolved cores; this is expected, because the parity and value tests are much weaker there, and it is not an error.

The engine of `semiprime47.cpp` also recovers a planted component with $b = 1$ through the divisor equation (`run47_output.txt`).

9.7 What is not independently verified

The following are *not* independently verified:

- the full-scale Step 1 for budgets 47 and 48, which was executed only by the C++ programs; the Lean function whose exhaustiveness is proved was executed for budgets 40 to 46 only (Section 9.5);
- the correctness of the compilers and the hardware;
- the informal correspondence of Section 7.1 between Computational Claims 6.4 and 7.1 and the source code, and the corresponding correspondence for Step 2 with budget 48.

The budget-46 Step 1 was, however, executed five times with identical results: in the recorded run, in a re-execution, through the engine of `semiprime47.cpp`, by `semiprime48.cpp`, and as the compiled Lean function. The budget-47 search was executed three times (twice without and once with (P4)), and all runs produced identical core dumps and identical solution lists (Section 9.3). The budget-48 search was executed twice, with identical core dumps (Section 8).

10 Lean formalization

The directory `lean/` contains a Lean 4 [3] development based on Mathlib [13]. It uses the toolchain `leanprover/lean4:v4.35.0-rc3` and the Mathlib commit

3cb72cfd416d1b5ec4b930d67648ef036a5df24f

and consists of 18 files with about 5,100 lines and about 320 theorems and lemmas: nine files for the analytic reduction and the solutions, and nine files for the search (`TopSum`, `Search`, `LossBound`, `SearchCorrect`, `SearchTables`, `SearchTablesProof`, `SearchReal`, `MainSearch`, `SearchSplit`). It contains no `sorry`, `admit`, `axiom` or `native_decide`. The main theorems depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`, as reported by `#print axioms (lean/AxiomsCheck.lean; output in paper/verification/lean_axioms_output.txt)`. Auto-bound implicit variables are disabled. The development builds with `lake build`.

Formally verified: the analytic reduction. The following results are proved in Lean without any computational hypothesis:

- the local criterion (Lemma 3.2; `integral_iff_localCond`);
- the exchange lemma, the list of the 47 smallest elements of $\mathcal{P}$ together with the fact that it is complete, the inequalities of Proposition 2.2, and Corollary 2.3 (`recipSum_le_Aupto`, `sp_le_158`, `H37_lt_one`, `H38_gt_one`, `H47_lt_two`, `recipSum_eq_one_of_integral`, `recipSum_eq_one_of_integral_48`, `card_ge_38`);
- Lemma 4.3 (`card_Uset_le`), Lemma 4.4 in the general form for every $U' \subseteq U$ (`value_lower_subset`), and all parts of Lemma 4.8 (`dcount_pos_of_mem_U`, `dcount_ne_one_of_not_mem_U`, `excess_le`, `parity_two`, `shape_value_bound`);
- the star lemmas and the star-size lemma (`star_dvd`, `star_card_ne_one`, `star_size`), the decomposition of G into stars and large-large edges (`bigPart_eq`, `recipSum_bigPart_eq`, `dcount_eq_mult`), and the finiteness of the candidate stars (`starCand_finite`);

- all parts of [Proposition 5.5](#) (`comp_value`, `comp_local_iff`, `comp_divisor_eq`, `comp_enum`, `comp_of_divisor`, `comp_pairs_finite`);
- the statement that the core and shape of every solution pass the tests of the search (`admissible_of_real`, `shapeOK_of_real`).

Unconditional verification of the solutions. `Sol23_valid` proves that each of the 23 sets consists of 47 squarefree semiprimes and has reciprocal sum 1, and `Sol23_card` proves that there are 23 distinct sets. The numerical facts are checked by the kernel through `decide` and `norm_num`. In particular, the existence part of [Theorem A](#) (sharpness) is fully formally verified.

The search, and the proof that it is exhaustive. Step 1 is written as a Lean function, and its exhaustiveness is a theorem.

- `Search.lean` defines an *instance* (the primes of $\mathcal{S}$, the budget, the least large prime, the list $v_1, v_2, \dots$ of [Section 6.1](#), θ , the share numerators and the number 8 of primes of the final step) and the search `Inst.search` as a recursive function over the levels (j, i) of [Algorithm 1](#). A core is a finite set of index pairs (a, b) , $a < b$, standing for $p_a p_b$. The tests (P1)–(P4) and the final test (4) are Boolean functions with exact natural-number and integer arithmetic. The expensive tables (VB, the maxima M of (P3), V_θ and the local bounds of (P4)) are an argument of the search.
- `SearchTables.lean` computes these tables with the formulas of the C++ program, the local bounds by a knapsack over residues (`dpRun`).
- `mem_search` (`SearchCorrect.lean`): for every instance and every family of tables with the soundness properties `TablesOK`, the output of the search contains every core that satisfies the rounded admissibility condition `IntAdm` (the budget, $\text{lo}(C) \leq 2^{96}$, and the rounded value test (4) for every upper segment of U). The proof follows the path to the core in the search tree and shows that no test fails there; for (P4) it uses `loss_bound` (`LossBound.lean`), the formal version of [Proposition 6.1](#), including the rounding of the shares.
- `mkTables_ok` (`SearchTablesProof.lean`): the computed tables satisfy `TablesOK`; in particular the knapsack computes upper bounds of the local maxima.
- `intAdm_of_admissible` (`SearchReal.lean`): for $\mathcal{S}$ = the primes up to 73 and $K \leq 48$ (the instance `inst73`), every core with `Admissible K C` ([Definition 4.5](#), with exact rationals) satisfies `IntAdm`. This includes the facts that the residues computed by the search are the residues ρ of the core, that $\text{lo}(n) \leq 2^{96}/n \leq \text{up}(n)$, and that the 49 listed numbers 158, 166, $\dots$, 471 are the 49 smallest elements of $\mathcal{P}$ with a large prime factor, so that $\text{VB}(x) \geq 2^{96}V(x)$.
- Hence `core_mem_search`: every admissible core (as its edge set `edgesOf C`) is in the output of `(inst73 K).search (inst73 K).mkTables`, for every $K \leq 48$; this is [Lemma 6.2](#). With `searchCores K` the list of cores of that output, `core_mem_searchCores` states that the core of every nonempty $T \subset \mathcal{P}$ with $|T| \leq K \leq 48$ and $\Sigma(T) \in \mathbb{Z}$ lies in `searchCores K`.

The search is thus a Lean function whose output is proved to contain the core of every solution; its output is not computed inside Lean’s kernel. The function is also executable, and the compiled function reproduces the output of the C++ program for budgets 40 to 46 and on reduced instances ([Section 9.5](#)).

Conditional completeness. Step 2 is not reproduced in Lean. Instead, [Computational Claims 6.4](#) and [7.1](#) are stated as the propositions `Search46` and `Search47` (reproduced in [Appendix B](#)). These are ordinary definitions used as hypotheses, not axioms. Their weaker versions `Search46'` and `Search47'` quantify only over the cores in `searchCores 46` and `searchCores 47`, the output of the verified search. By the exhaustiveness theorem, `Search46'` implies `Search46` and `Search47'` implies `Search47` (`search46_of`, `search47_of`). Lean proves:

- `no_integral_sum_le_46`, which is [Theorem A](#) with `Search46` as a hypothesis, its reformulation `card_ge_47_of_integral`, and the same statements with the hypothesis `Search46'` (`no_integral_sum_le_46'`, `card_ge_47_of_integral'`);
- `solutions47_iff`, which is [Theorem B](#) with `Search47` as a hypothesis; `solutions47_integral_iff` states the version for integral sums, and `solutions47_ncard` states that the set of all 47-term representations of 1 has exactly 23 elements; the primed versions use the hypothesis `Search47'`.

Consequently the complete proofs of [Theorems A](#) and [B](#) are formally verified *modulo* `Search46'` and `Search47'`: the enumeration of the cores is no longer an assumption, and what remains assumed is the outcome of Step 2 on the cores output by the verified search. These hypotheses are established by the computations of [Sections 6](#) and [7](#). The following points are not verified in Lean:

- the fact that the C++ Step 1 outputs (at least) the cores of `searchCores K`; for $K = 46$ this was checked by running the compiled Lean function, and for $K = 47$ it rests on the C++ program implementing the same function ([Section 9.5](#));
- the fact that the C++ Step 2 implements [Computational Claims 6.4](#) and [7.1](#) correctly ([Section 7.1](#));
- the fact that the recorded runs produced the stated output.

For [Theorem C](#), Lean provides the reduction to the cores of `searchCores 48` (`core_mem_searchCores`), but the Step 2 statement for budget 48 (in particular the case $b = 2$ of [Section 8.1](#)) is not formalized, and the 620 sets are not verified in Lean.

While the first part of the development was being written, the formalization exposed the two corrections of [Remarks 4.6](#) and [5.6](#). The Step 1 hypothesis is stated with the upper-segment condition of [Definition 4.5](#). With only the final value test, it would not describe what the search checks.

11 Computational reproducibility

Arithmetic. None of the C++ programs uses a floating-point type (`float`, `double` or `long double`), nor any floating-point library function.

- Congruences are computed with machine integers modulo primes at most 73.
- Exact rational comparisons (the values H_k , the verification of solutions, the exact value tests of covers) use either a small arbitrary-precision natural-number type or 128-bit integers over the common denominator $D = \prod_{p \in \mathcal{S}} p \approx 4.0 \cdot 10^{28}$.
- Bounds on sums of reciprocals use the 96-bit fixed-point numbers up and lo of [Section 6.1](#). Upper bounds are always formed from up and lower bounds from lo. Every pruning decision compares an upper bound of a quantity that must be at least 1 with 2^{96} , or a lower bound of a quantity that must be at most 1 with 2^{96} .

- The loss bound (P4) uses signed 128-bit integers; the shares are rounded up and the knapsack tables are exact maxima, so it is an upper bound as in Proposition 6.1.

Rounding can therefore only make a test *weaker*. It may keep a core or shape that exact arithmetic would discard, and this costs only time; it can never discard an admissible core or a feasible shape. All fixed-point sums are below $3 \cdot 2^{96} < 2^{128}$. The Python program uses exact rationals throughout.

Primality and factorization. The earlier programs decide primality of 64-bit integers with the Miller–Rabin test [11] to the bases $2, 3, 5, \dots, 37$ (the first 12 primes). This test is deterministic for all $n < \psi_{12} = 318\,665\,857\,834\,031\,151\,167\,461 \approx 3.2 \cdot 10^{23}$ [12], and in particular for all $n < 2^{64}$. The $b = 1$ branch of `semiprime47.cpp` contains a probabilistic test for integers of at least 2^{64} . It was never used: the run reports 0 probable primes. `semiprime48.cpp` uses the first 13 prime bases, which give a deterministic test below $\psi_{13} \approx 3.3 \cdot 10^{24}$ [12], and counts every number above that bound that passes the test as a probable prime; all its runs report 0 probable primes. Factorizations use trial division and Pollard’s rho method [10]. The correctness of the result does not depend on the heuristics of the method, because every returned factor is certified prime and the factors multiply to the input by construction; only termination does, and the runs terminated. The inverses modulo p are computed as $a^{p-2} \bmod p$.

Build and run. The programs need a C++11 compiler with `unsigned __int128` (GCC or Clang):

```
g++ -O3 -march=native -pthread -o semiprime48 semiprime48.cpp
./semiprime48 4 --budget 46           # Theorem A, 4 threads (~2 s)
./semiprime48 4 --budget 47           # Theorem B (~1 min)
./semiprime48 4 --selftest --dump cores48.txt # Theorem C (~30 min)
python3 verify48.py run48_output.txt
gunzip -k cores48.txt.gz && python3 complete48.py cores48.txt 48 4
bash validation48/step1_equivalence.sh 4 # old vs new Step 1
# earlier programs:
g++ -O3 -march=native -pthread -o semiprime_reciprocals semiprime_reciprocals.cpp
./semiprime_reciprocals 4             # budget 46
g++ -O3 -march=native -pthread -o semiprime47 semiprime47.cpp
./semiprime47 4 --selftest --dump cores47.txt
gunzip -k cores47.txt.gz && python3 complete47.py cores47.txt 47
```

The recorded runs of `semiprime48.cpp` used 4 threads and took 2.1 s (budget 46), 65 s (budget 47) and 1770 s (budget 48, 29 min 30 s) for search and completion. The recorded runs of the earlier programs also used 4 threads. The budget-46 search took 1179 s (19 min 40 s wall-clock), and the budget-47 run took 20 898 s for the search and 5 h 48 min wall-clock in total. The Lean search is compiled to native code with the scripts in `validation48/lean_runner/`, and `validation48/lean_vs_cpp.sh` compares its output with that of `semiprime48.cpp`. The resident memory of the search is only a few megabytes; the virtual size of about 300 MB reported in the repository documentation consists mostly of thread stacks. The recorded outputs do not state the processor model. The re-executions of Section 9.3 used a cloud container with 4 virtual CPUs (Intel Xeon, 2.10 GHz) and GCC 13.3.0. There the budget-47 search took 19 388 s with 4 threads.

Files. The repository (*<repository URL to be inserted>*) contains the following files.

- Programs: `semiprime48.cpp`, `complete48.py` and `verify48.py`; the earlier programs `semiprime_reciprocals.cpp`, `semiprime47.cpp` and `complete47.py`.

- Recorded outputs:
 - of `semiprime48.cpp`: `run48_output.txt`, `validation48/run46_output.txt` and `validation48/run47_output.txt`;
 - of the Python checks: `run48_python_crosscheck.txt`, `verify48_output.txt` and `run47_python_crosscheck.txt`;
 - of the earlier programs: `proof_run_output.txt`, `selftest_output.txt` and `run47_output.txt`.
- Data: the core dumps `cores47.txt.gz` and `cores48.txt.gz`, and the lists `solutions47.txt` and `solutions48.txt`.
- Validation of the new search: `validation48/` (old versus new Step 1, brute force, the case $b = 2$, Lean versus C++).
- The Lean development `lean/`.
- The files of this paper in `paper/`, including the verification scripts, the brute-force test and the logs of the re-execution (`paper/verification/`).

12 Discussion

We have shown that at least 47 distinct squarefree semiprimes are needed to obtain an integral sum of reciprocals, and that exactly 23 sets of 47 such numbers have reciprocal sum 1. With the same method we found that exactly 620 sets of 48 such numbers have reciprocal sum 1. The minimality result confirms Watanabe’s conjecture [14].

The main mathematical point is the finite reduction. The local criterion (Lemma 3.2) turns integrality into a condition at each vertex of a graph. The core decomposition bounds the number of unsatisfied small primes. The star lemma and the component equation bound the large primes by divisibility, not by size. That large primes occur, for example 3779, shows that a bound of this kind is unavoidable. A single large–large edge is handled by an explicit factorization identity, and two such edges by a size argument that bounds one of the large primes (Section 8.1). Configurations with three or more such edges are excluded here only by the computation: no feasible shape with $b \geq 3$ exists for budget 48. For larger budgets this case would need further ideas. On the computational side, the loss bound (Section 6.2) makes Step 1 thousands of times faster. It combines a Lagrangian relaxation of the budget constraint with the congruences that every prime not yet processed must eventually satisfy, and it leaves the output of the search unchanged.

Limitations. The proofs are computer-assisted.

- Step 1 is now a Lean function with a formal proof of exhaustiveness. The remaining gap is that the full-scale executions for budgets 47 and 48 were made by the C++ program, which is meant to compute the same function but is not verified; on every instance on which both were run, including the real instance with budgets 40 to 46, the two produce the same cores. The two generations of C++ programs share most of their Step 1 code.
- Step 2 has been implemented twice, in C++ and in Python, and the implementations agree for all three budgets.
- The reduction, the exhaustiveness of Step 1 and the 23 solutions are verified in Lean. The connection between the Lean hypotheses `Search46’` and `Search47’` and the C++ Step 2

rests on the informal argument of [Section 7.1](#). For budget 48, Step 2 (including the case $b = 2$) and the 620 solutions are not formalized.

A fully formal proof would require the computation to emit a certificate that can be checked in Lean, such as the list of cores output by Step 1 together with a refutation or completion of each feasible shape; alternatively, Step 2 could itself be written and verified as a Lean function, as was done here for Step 1, and both executed in a kernel-checkable way. This is a natural direction for future work. An independent reimplementaion of the search, preferably by other authors and with a different enumeration strategy, would also strengthen the result considerably.

Further questions. The same method applies to other targets and budgets. Examples include the minimal number of terms for the integer 2, or for denominators with exactly three distinct prime factors (compare [1]). With the loss bound the cost of Step 1 still grows by a factor of about 20 per unit of budget: $2.6 \cdot 10^7$, $6.0 \cdot 10^8$ and $1.3 \cdot 10^{10}$ nodes for budgets 46, 47 and 48.

13 Disclosure of the use of generative AI

Generative AI tools, specifically Claude Code (Anthropic), an AI coding assistant based on large language models, were used substantially throughout this project, and not only for preparing the manuscript. Under the direction of the authors, the AI tool developed the mathematical reduction presented here, including the loss bound of [Section 6.2](#) and the treatment of two large-large elements in [Section 8.1](#), and it wrote the following:

- the C++ programs and the Python cross-checks;
- the Lean formalization, including the search as a Lean function and the proof of its exhaustiveness;
- the documentation files of the repository;
- the verification scripts written for this paper;
- a first draft of this manuscript and of its revision.

The exploratory mathematical reasoning was also carried out with the tool. The two corrections of [Remarks 4.6](#) and [5.6](#) were found during the AI-assisted formalization and a later audit of the code.

All mathematical claims in this article were checked by the verification procedures described in [Sections 8 to 10](#):

- exact computation;
- second implementations of Step 2;
- brute-force validation of Step 1 on reduced instances, and the comparison of the two generations of the search;
- a Lean 4 formalization of the analytic reduction, of the exhaustiveness of Step 1 and of the 23 solutions, and the execution of the verified Lean search for budgets 40 to 46.

These checks were themselves designed and executed with the assistance of the same tool. The extent of the computer-assisted part and of the formally verified part, and the exact status of every result, are stated explicitly in [Figure 1](#) and [Sections 6 to 10](#). *[Editable by the authors: describe here any independent human checking of the code, the computations and the proofs]*

that has been carried out.] No AI system is an author of this work. The authors take full responsibility for its content.

Acknowledgements

References

- [1] Steve Butler, Paul Erdős, and Ron Graham. Egyptian fractions with each denominator having three distinct prime divisors. *Integers*, 15:Paper No. A51, 2015.
- [2] Agustina Czenky, Emily McGovern, Julia Plavnik, Eric Rowell, and Abigail Watkins. Egyptian fractions for few primes, 2025. arXiv:2507.03727 [math.NT], preprint.
- [3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [4] P. Erdős and R. L. Graham. *Old and New Problems and Results in Combinatorial Number Theory*, volume 28 of *Monographies de L’Enseignement Mathématique*. Université de Genève, Geneva, 1980.
- [5] R. L. Graham. On finite sums of unit fractions. *Proc. London Math. Soc.* (3), 14(2): 193–207, 1964. doi: 10.1112/plms/s3-14.2.193.
- [6] Ronald L. Graham. Paul Erdős and Egyptian fractions. In László Lovász, Imre Z. Ruzsa, and Vera T. Sós, editors, *Erdős Centennial*, volume 25 of *Bolyai Society Mathematical Studies*, pages 289–309. Springer, Berlin, Heidelberg, 2013. doi: 10.1007/978-3-642-39286-3_9.
- [7] Richard K. Guy. *Unsolved Problems in Number Theory*. Problem Books in Mathematics. Springer, New York, third edition, 2004.
- [8] Allan Wm. Johnson. *Crux Mathematicorum*, 4:190, 1978. Bibliographic details not verified against the original (see AUDIT_REPORT.md).
- [9] Shisheng Li. Every natural number is a sum of distinct semiprime unit fractions, 2026. arXiv:2606.15159 [math.NT], preprint.
- [10] J. M. Pollard. A Monte Carlo method for factorization. *BIT*, 15(3):331–334, 1975. doi: 10.1007/BF01933667.
- [11] Michael O. Rabin. Probabilistic algorithm for testing primality. *J. Number Theory*, 12(1): 128–138, 1980. doi: 10.1016/0022-314X(80)90084-0.
- [12] Jonathan Sorenson and Jonathan Webster. Strong pseudoprimes to twelve prime bases. *Math. Comp.*, 86(304):985–1003, 2017. arXiv:1509.00864.
- [13] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.
- [14] Tatsuru Watanabe. New examples of the representation of 1 by the sum of reciprocals of semiprime numbers, 2020. arXiv:2009.03275 [math.NT].

A The 23 representations of 1 with 47 terms

The 23 sets $T \subset \mathcal{P}$ with $|T| = 47$ and $\Sigma(T) = 1$, in the order of `solutions47.txt`. Solutions 1, 2, 3, 5, 11 and 12 contain a prime larger than 73 (Table 2).

1. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 87, 91, 93, 95, 111, 115, 119, 123, 133, 143, 145, 155, 203, 219, 221, 287, 299, 391, 481, 1299, 2117, 16021, 31609
2. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 87, 91, 93, 95, 111, 115, 119, 123, 133, 143, 155, 185, 203, 221, 287, 299, 319, 391, 407, 481, 933, 1555, 11507
3. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 87, 91, 93, 95, 111, 115, 119, 123, 133, 145, 155, 161, 187, 221, 253, 259, 287, 407, 629, 667, 1509, 6539, 14587
4. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 85, 87, 91, 93, 95, 115, 119, 123, 129, 133, 143, 145, 155, 187, 213, 221, 287, 301, 559, 629, 781, 1247, 1591, 1633
5. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 86, 87, 91, 93, 95, 106, 111, 115, 123, 133, 155, 159, 161, 185, 203, 215, 253, 265, 583, 667, 731, 109591, 139823, 154939
6. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 82, 86, 91, 93, 95, 111, 119, 122, 123, 133, 143, 145, 155, 161, 183, 187, 259, 287, 299, 319, 473, 481, 559, 671, 1591
7. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 85, 86, 87, 91, 93, 94, 95, 111, 115, 119, 133, 141, 142, 145, 155, 187, 213, 517, 629, 799, 1247, 1591, 1633, 2021, 2627
8. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 77, 86, 87, 91, 93, 94, 95, 111, 115, 118, 119, 129, 133, 145, 155, 161, 185, 187, 329, 517, 667, 851, 1073, 1357, 1363, 2537
9. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 74, 82, 85, 86, 87, 91, 93, 95, 106, 111, 123, 133, 145, 155, 159, 185, 203, 215, 253, 265, 287, 319, 493, 583, 731, 851, 1073
10. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 111, 115, 119, 133, 141, 142, 155, 187, 205, 213, 235, 493, 517, 1363, 1633, 1763, 1927, 2627
11. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 115, 118, 119, 123, 129, 133, 141, 155, 203, 287, 319, 391, 799, 1345, 1357, 2537, 2959, 12643
12. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 115, 118, 119, 123, 133, 143, 145, 155, 203, 221, 287, 391, 559, 1357, 2537, 12209, 18103, 19787
13. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 115, 119, 123, 129, 133, 141, 142, 145, 155, 187, 517, 799, 1207, 1247, 1633, 1763, 2021, 2911
14. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 118, 119, 123, 129, 133, 155, 161, 187, 213, 287, 329, 355, 493, 497, 517, 1357, 1363, 2537
15. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 91, 93, 94, 95, 118, 119, 123, 133, 145, 155, 161, 187, 203, 215, 287, 329, 493, 517, 1247, 1357, 1363, 2537
16. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 93, 94, 95, 115, 118, 119, 123, 129, 133, 141, 143, 145, 155, 287, 377, 391, 551, 799, 893, 1357, 1363, 2537
17. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 87, 93, 94, 95, 115, 118, 119, 123, 129, 133, 143, 145, 155, 177, 287, 299, 391, 551, 611, 799, 893, 1711, 2537
18. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 86, 91, 93, 94, 95, 115, 119, 123, 133, 141, 142, 143, 155, 203, 221, 235, 287, 299, 355, 377, 391, 559, 2021, 2059
19. 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 85, 87, 91, 93, 94, 95, 111, 115, 118, 119, 123, 133, 142, 155, 187, 213, 287, 295, 329, 413, 493, 517, 1363, 1633, 2627

- 20.** 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 86, 87, 91, 93, 94, 95, 118, 119, 123, 129, 133, 145, 155, 161, 177, 187, 203, 287, 299, 329, 377, 517, 1363, 1711, 2537
- 21.** 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 86, 91, 93, 94, 95, 111, 115, 119, 123, 133, 141, 142, 143, 145, 155, 203, 213, 221, 235, 287, 493, 559, 1633, 2021, 2627
- 22.** 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 69, 77, 82, 87, 91, 93, 95, 106, 111, 115, 119, 122, 123, 133, 142, 155, 159, 183, 187, 203, 213, 265, 287, 319, 583, 671, 1633, 2627
- 23.** 6, 10, 14, 15, 21, 22, 26, 33, 34, 35, 38, 39, 46, 51, 55, 57, 58, 62, 65, 74, 77, 82, 85, 87, 91, 93, 95, 111, 115, 119, 123, 129, 133, 143, 145, 155, 161, 221, 253, 259, 287, 299, 391, 473, 481, 1247, 1591

B The computational hypotheses in Lean

The exhaustiveness theorem of the search and the hypotheses of [Section 10](#), verbatim from `lean/SemiprimeEgypt/SearchReal.lean`, `MainSearch.lean` and `Main.lean`. The primed hypotheses `Search46'` and `Search47'` are obtained from `Search46` and `Search47` by restricting every quantifier over cores `C` to the cores in `searchCores K`, the output of the verified search; `Search46'` is shown below, and `Search47'` is `Search47` with $\forall C \in \text{searchCores } 47$, in place of $\forall C : \text{Finset } \mathbb{N}$, in each of its four parts.

- `CoreSet C` says that every element of C lies in $\mathcal{P}_S$.
- `Admissible K C` is [Definition 4.5](#).
- `ShapeOK K C d b` is [Definition 4.7](#).
- `StarCand m C d P A` is [Definition 5.3](#).
- `mult`, `starEdges` and `starVal` are the multiplicities, elements and value of a star family.
- `compN` and `compP\(_1\)`, `compP\(_2\)` are N , P_1 and P_2 of [Proposition 5.5](#).
- `Sol23` is the family of [Appendix A](#).

```

theorem core_mem_search {K : ℕ} (hK : K ≤ 48) {C : Finset ℕ} (hC : CoreSet C)
  (hA : Admissible K C) : edgesOf C ∈ (inst73 K).search (inst73 K).mkTables

def searchCores (K : ℕ) : List (Finset ℕ) :=
  ((inst73 K).search (inst73 K).mkTables).map coreOf

theorem core_mem_searchCores {T : Finset ℕ} (hT : ∀ n ∈ T, IsSP n) (hne : T.Nonempty)
  {K : ℕ} (hK : K ≤ 48) (hc : T.card ≤ K) (hz : ∃ z : ℤ, recipSum T = z) :
  core 73 T ∈ searchCores K

def Search46' : Prop :=
  ∀ C ∈ searchCores 46, CoreSet C → Admissible 46 C →
    (Uset 73 C).Nonempty ∧
    ∀ (d : ℕ → ℕ) (b : ℕ), ShapeOK 46 C d b → b = 0 ∧ ∀ P A, ¬ StarCand 10 C d P A

def Search46 : Prop :=
  ∀ C : Finset ℕ, CoreSet C → Admissible 46 C →
    (Uset 73 C).Nonempty ∧
    ∀ (d : ℕ → ℕ) (b : ℕ), ShapeOK 46 C d b → b = 0 ∧ ∀ P A, ¬ StarCand 10 C d P A

def Search47 : Prop :=
  (∀ C : Finset ℕ, CoreSet C → Admissible 47 C → Uset 73 C = ∅ → C.Nonempty → C ∈ Sol23) ∧
  (∀ C : Finset ℕ, CoreSet C → Admissible 47 C → (Uset 73 C).Nonempty →
    ∀ (d : ℕ → ℕ) (b : ℕ), ShapeOK 47 C d b → b ≤ 1) ∧
  (∀ (C : Finset ℕ) (d : ℕ → ℕ) (Ps : Finset ℕ) (A : ℕ → Finset ℕ),
    CoreSet C → Admissible 47 C → (Uset 73 C).Nonempty → ShapeOK 47 C d 0 →
    (∀ P ∈ Ps, StarCand 11 C d P (A P)) →

```

```

(∀ q ∈ smallPrimes 73, mult Ps A q = d q) →
recipSum C + starVal Ps A = 1 →
C ∪ starEdges Ps A ∈ Sol23) ∧
(∀ (C : Finset N) (d : N → N) (Ps : Finset N) (A : N → Finset N) (A1 A2 : Finset N)
(t X : N),
CoreSet C → Admissible 47 C → (Uset 73 C).Nonempty → ShapeOK 47 C d 1 →
(∀ P ∈ Ps, StarCand 11 C d P (A P)) →
A1.Nonempty → A2.Nonempty → A1 ⊆ smallPrimes 73 → A2 ⊆ smallPrimes 73 →
(∀ q ∈ smallPrimes 73,
mult Ps A q + (if q ∈ A1 then 1 else 0) + (if q ∈ A2 then 1 else 0) = d q) →
recipSum C + starVal Ps A < 1 →
0 < t →
(t : ℚ) = (1 - recipSum C - starVal Ps A) * (∏q ∈ A1 (q : ℚ)) * ∏q ∈ A2 (q : ℚ) →
0 < X → X | compN A1 A2 t → t | X + nA A1 * ∏q ∈ A2 q →
t | compN A1 A2 t / X + nA A2 * ∏q ∈ A1 q →
compP1 A1 A2 t X ≠ compP2 A1 A2 t X →
73 < compP1 A1 A2 t X → 73 < compP2 A1 A2 t X →
(compP1 A1 A2 t X).Prime → (compP2 A1 A2 t X).Prime →
compP1 A1 A2 t X ∉ Ps → compP2 A1 A2 t X ∉ Ps →
C ∪ starEdges Ps A ∪ A1.image (· * compP1 A1 A2 t X) ∪
A2.image (· * compP2 A1 A2 t X) ∪ {compP1 A1 A2 t X * compP2 A1 A2 t X} ∈ Sol23)

```

Table 4: The 23 solutions of [Theorem B](#). Each solution consists of the 23 common elements listed in the text together with the 24 elements in the last column. “Core” is the number of elements with both prime factors at most 73.

no.	max. prime	core	$P > 73$	remaining 24 elements
1	433	44	433	69, 74, 77, 82, 85, 87, 91, 111, 115, 119, 123, 143, 145, 203, 219, 221, 287, 299, 391, 481, 1299, 2117, 16021, 31609
2	311	44	311	69, 74, 77, 82, 85, 87, 91, 111, 115, 119, 123, 143, 185, 203, 221, 287, 299, 319, 391, 407, 481, 933, 1555, 11507
3	503	44	503	69, 74, 77, 82, 85, 87, 91, 111, 115, 119, 123, 145, 161, 187, 221, 253, 259, 287, 407, 629, 667, 1509, 6539, 14587
4	71	47	–	69, 74, 77, 82, 85, 87, 91, 115, 119, 123, 129, 143, 145, 187, 213, 221, 287, 301, 559, 629, 781, 1247, 1591, 1633
5	3779	44	3779	69, 74, 77, 82, 86, 87, 91, 106, 111, 115, 123, 159, 161, 185, 203, 215, 253, 265, 583, 667, 731, 109591, 139823, 154939
6	61	47	–	69, 74, 77, 82, 86, 91, 111, 119, 122, 123, 143, 145, 161, 183, 187, 259, 287, 299, 319, 473, 481, 559, 671, 1591
7	71	47	–	69, 74, 77, 85, 86, 87, 91, 94, 111, 115, 119, 141, 142, 145, 187, 213, 517, 629, 799, 1247, 1591, 1633, 2021, 2627
8	59	47	–	69, 74, 77, 86, 87, 91, 94, 111, 115, 118, 119, 129, 145, 161, 185, 187, 329, 517, 667, 851, 1073, 1357, 1363, 2537
9	53	47	–	69, 74, 82, 85, 86, 87, 91, 106, 111, 123, 145, 159, 185, 203, 215, 253, 265, 287, 319, 493, 583, 731, 851, 1073
10	71	47	–	69, 77, 82, 85, 86, 87, 91, 94, 111, 115, 119, 141, 142, 187, 205, 213, 235, 493, 517, 1363, 1633, 1763, 1927, 2627
11	269	44	269	69, 77, 82, 85, 86, 87, 91, 94, 115, 118, 119, 123, 129, 141, 203, 287, 319, 391, 799, 1345, 1357, 2537, 2959, 12643
12	421	44	421	69, 77, 82, 85, 86, 87, 91, 94, 115, 118, 119, 123, 143, 145, 203, 221, 287, 391, 559, 1357, 2537, 12209, 18103, 19787
13	71	47	–	69, 77, 82, 85, 86, 87, 91, 94, 115, 119, 123, 129, 141, 142, 145, 187, 517, 799, 1207, 1247, 1633, 1763, 2021, 2911
14	71	47	–	69, 77, 82, 85, 86, 87, 91, 94, 118, 119, 123, 129, 161, 187, 213, 287, 329, 355, 493, 497, 517, 1357, 1363, 2537
15	59	47	–	69, 77, 82, 85, 86, 87, 91, 94, 118, 119, 123, 145, 161, 187, 203, 215, 287, 329, 493, 517, 1247, 1357, 1363, 2537
16	59	47	–	69, 77, 82, 85, 86, 87, 94, 115, 118, 119, 123, 129, 141, 143, 145, 287, 377, 391, 551, 799, 893, 1357, 1363, 2537
17	59	47	–	69, 77, 82, 85, 86, 87, 94, 115, 118, 119, 123, 129, 143, 145, 177, 287, 299, 391, 551, 611, 799, 893, 1711, 2537
18	71	47	–	69, 77, 82, 85, 86, 91, 94, 115, 119, 123, 141, 142, 143, 203, 221, 235, 287, 299, 355, 377, 391, 559, 2021, 2059
19	71	47	–	69, 77, 82, 85, 87, 91, 94, 111, 115, 118, 119, 123, 142, 187, 213, 287, 295, 329, 413, 493, 517, 1363, 1633, 2627
20	59	47	–	69, 77, 82, 86, 87, 91, 94, 118, 119, 123, 129, 145, 161, 177, 187, 203, 287, 299, 329, 377, 517, 1363, 1711, 2537
21	71	47	–	69, 77, 82, 86, 91, 94, 111, 115, 119, 123, 141, 142, 143, 145, 203, 213, 221, 235, 287, 493, 559, 1633, 2021, 2627
22	71	47	–	69, 77, 82, 87, 91, 106, 111, 115, 119, 122, 123, 142, 159, 183, 187, 203, 213, 265, 287, 319, 583, 671, 1633, 2627
23	43	47	–	74, 77, 82, 85, 87, 91, 111, 115, 119, 123, 129, 143, 145, 161, 221, 253, 259, 287, 299, 391, 473, 481, 1247, 1591

Table 5: The 620 sets $T \subset \mathcal{P}$ with $|T| = 48$ and $\Sigma(T) = 1$, by the primes larger than 73 that they contain (`solutions48.txt`; recomputed by `paper/verification/verify48_facts.py`).

primes larger than 73	number of sets
none	397
one prime P , the centre of a star	187
two primes, both centres of stars	18
two primes P_1, P_2 with $P_1 P_2 \in T$	18
total	620

Table 6: Brute-force validation of Step 1 on reduced instances ($\mathcal{S}$ = primes up to s). D is the output of the search, with the default table of the 8 smallest primes and with a table of 3 primes. The output did not depend on the number of threads. In all cases $\text{SEG} \subseteq D \subseteq \text{FIN}$.

s	edges	budget	SEG	D , table size		FIN
				8	3	
13	15	40	13	13	13	140
13	15	45	156	161	161	512
13	15	50	539	586	586	1 284
13	15	60	2 083	2 390	2 390	3 872
13	15	80	7 260	8 492	8 492	10 774
17	21	40	32	32	32	484
17	21	45	2 388	2 422	2 422	10 001
17	21	50	13 111	13 669	13 669	36 845
17	21	60	72 019	79 084	79 083	139 032
19	28	40	20	20	20	2 535
19	28	45	38 109	38 109	38 109	175 804
19	28	50	475 652	478 084	478 084	1 146 681